

RTCA, Inc.
1150 18th Street, NW, Suite 910
Washington, D.C. 20036

Formal Methods Supplement to DO-178C and DO-278A

RTCA DO-333
December 13, 2011

Prepared by: SC-205
© 2011 RTCA, Inc.

Copies of this document may be obtained from

RTCA, Inc.

Telephone: 202-833-9339

Facsimile: 202-833-9434

Internet: www.rtca.org

Please visit the RTCA Online Store for document pricing and ordering information.

FOREWORD

This report was prepared by RTCA Special Committee 205 (SC-205) and EUROCAE Working Group 71 (WG-71) and approved by the RTCA Program Management Committee (PMC) on December 13, 2011.

RTCA, Incorporated is a not-for-profit corporation formed to advance the art and science of aviation and aviation electronic systems for the benefit of the public. The organization functions as a Federal Advisory Committee and develops consensus-based recommendations on contemporary aviation issues. RTCA's objectives include but are not limited to:

- coalescing aviation system user and provider technical requirements in a manner that helps government and industry meet their mutual objectives and responsibilities;
- analyzing and recommending solutions to the system technical issues that aviation faces as it continues to pursue increased safety, system capacity, and efficiency;
- developing consensus on the application of pertinent technology to fulfill user and provider requirements, including development of minimum operational performance standards for electronic systems and equipment that support aviation; and
- assisting in developing the appropriate technical material upon which positions for the International Civil Aviation Organization and the International Telecommunication Union and other appropriate international organizations can be based.

The organization's recommendations are often used as the basis for government and private sector decisions as well as the foundation for many Federal Aviation Administration Technical Standard Orders.

Since the RTCA is not an official agency of the United States Government, its recommendations may not be regarded as statements of official government policy unless so enunciated by the U.S. government organization or agency having statutory jurisdiction over any matters to which the recommendations relate.

This Page Intentionally Left Blank

TABLE OF CONTENTS

FM.1.0	INTRODUCTION	1
FM.1.1	PURPOSE.....	2
FM.1.2	SCOPE.....	2
FM.1.3	RELATIONSHIP TO OTHER DOCUMENTS.....	2
FM.1.4	HOW TO USE THIS DOCUMENT	2
FM.1.5	DOCUMENT OVERVIEW	3
FM.1.6	CHARACTERISTICS OF FORMAL METHODS	3
FM.1.6.1	Formal Models	3
FM.1.6.2	Formal Analysis	4
FM.1.7	FORMAL METHODS AS A VERIFICATION TECHNIQUE.....	5
FM.2.0	SYSTEM ASPECTS RELATING TO SOFTWARE DEVELOPMENT.....	7
FM.3.0	SOFTWARE LIFE CYCLE	9
FM.4.0	SOFTWARE PLANNING PROCESS.....	11
FM.4.3	SOFTWARE PLANS	11
FM.5.0	SOFTWARE DEVELOPMENT PROCESSES.....	13
FM.6.0	SOFTWARE VERIFICATION PROCESS	15
FM.6.1	PURPOSE OF SOFTWARE VERIFICATION	15
FM.6.2	OVERVIEW OF SOFTWARE VERIFICATION PROCESS ACTIVITIES	17
FM.6.2.1	Considerations for Formal Methods	17
FM.6.3	SOFTWARE REVIEWS AND ANALYSES	17
FM.6.3.1	Reviews and Analyses of High-Level Requirements	18
FM.6.3.2	Reviews and Analyses of Low-Level Requirements	19
FM.6.3.3	Reviews and Analyses of Software Architecture.....	20
FM.6.3.4	Reviews and Analyses of Source Code	21
FM.6.3.5	Reviews and Analyses of the Outputs of the Integration Process	22
FM.6.3.6	Reviews and Analyses of the Formal Analysis Cases, Procedures, and Results for Requirements, Architecture, and Source Code	22
FM.6.4	SOFTWARE TESTING	22
FM.6.5	SOFTWARE VERIFICATION PROCESS TRACEABILITY.....	23
FM.6.6	VERIFICATION OF PARAMETER DATA ITEMS	23
FM.6.7	FORMAL ANALYSIS OF THE EXECUTABLE OBJECT CODE	23
FM.6.7.1	Principles of Coverage Analysis When Using Formal Methods.....	28
FM.6.7.2	Reviews and Analyses of the Formal Analysis Cases, Procedures, and Results for the Executable Object Code ..	30
FM.7.0	SOFTWARE CONFIGURATION MANAGEMENT PROCESS.....	9
FM.8.0	SOFTWARE QUALITY ASSURANCE PROCESS.....	9
FM.9.0	CERTIFICATION LIAISON PROCESS	9
FM.10.0	OVERVIEW OF CERTIFICATION PROCESS	9

FM.11.0	SOFTWARE LIFE CYCLE DATA.....	9
FM.11.1	PLAN FOR SOFTWARE ASPECTS OF CERTIFICATION	41
FM.11.3	SOFTWARE VERIFICATION PLAN	41
FM.11.6	SOFTWARE REQUIREMENTS STANDARDS	41
FM.11.7	SOFTWARE DESIGN STANDARDS	42
FM.11.8	SOFTWARE CODE STANDARDS	42
FM.11.13	SOFTWARE VERIFICATION CASES AND PROCEDURES	42
FM.12.0	ADDITIONAL CONSIDERATIONS	43
FM.12.3.5	Coverage Analysis When Using a Combination of Formal Methods and Testing	43
ANNEX FM.A	– PROCESS OBJECTIVES AND OUTPUTS BY SOFTWARE LEVEL IN DO-178C.....	45
ANNEX FM.B	– ACRONYMS AND GLOSSARY OF TERMS.....	41
ANNEX FM.C	– PROCESS OBJECTIVES AND OUTPUTS BY ASSURANCE LEVEL IN DO-278A.....	43
APPENDIX FM.A	– COMMITTEE MEMBERSHIP	9
APPENDIX FM.B	– FREQUENTLY ASKED QUESTIONS AND DISCUSSION PAPERS.....	1
FM.B.1	DP #1: EXAMPLES OF FORMAL METHODS	1

LIST OF FIGURES

FIGURE FM.6-1	LEVEL A SOFTWARE VERIFICATION PROCESSES	16
FIGURE FM.6-2	POSSIBLE VERIFICATION PATHS FOR EXECUTABLE OBJECT CODE WITH RESPECT TO SOURCE CODE AND REQUIREMENTS.....	28
FIGURE FM.B.1-1	SOFTWARE LIFE CYCLE FOR UNIT PROOF	B-8
FIGURE FM.B.1-2	PROOF PROCESS	B-13
FIGURE FM.B.1-3	OVERVIEW OF REQUIREMENT ARTIFACTS.....	B-19
FIGURE FM.B.1-4	SIMPLIFIED DISPLAY SYSTEM ARCHITECTURE	B-20
FIGURE FM.B.1-5	WINDOW MANAGER TOP LEVEL SOFTWARE MODEL	B-24
FIGURE FM.B.1-6	ANALYSIS PROCESS	B-25

LIST OF TABLES

TABLE FM.A-1	SOFTWARE PLANNING PROCESS.....	46
TABLE FM.A-2	SOFTWARE DEVELOPMENT PROCESSES	47
TABLE FM.A-3	VERIFICATION OF OUTPUTS OF SOFTWARE REQUIREMENTS PROCESS	48
TABLE FM.A-4	VERIFICATION OF OUTPUTS OF SOFTWARE DESIGN PROCESS	49
TABLE FM.A-5	VERIFICATION OF OUTPUTS OF SOFTWARE CODING & INTEGRATION PROCESSES.....	50
TABLE FM.A-6	TESTING OF OUTPUTS OF INTEGRATION PROCESS.....	51
TABLE FM.A-7	VERIFICATION OF VERIFICATION PROCESS RESULTS	52
TABLE FM.A-8	SOFTWARE CONFIGURATION MANAGEMENT PROCESS.....	53
TABLE FM.A-9	SOFTWARE QUALITY ASSURANCE PROCESS.....	54
TABLE FM.A-10	CERTIFICATION LIAISON PROCESS	55
TABLE FM.C-1	SOFTWARE PLANNING PROCESS.....	44
TABLE FM.C-2	SOFTWARE DEVELOPMENT PROCESSES	45
TABLE FM.C-3	VERIFICATION OF OUTPUTS OF SOFTWARE REQUIREMENTS PROCESS	46
TABLE FM.C-4	VERIFICATION OF OUTPUTS OF SOFTWARE DESIGN PROCESS	47
TABLE FM.C-5	VERIFICATION OF OUTPUTS OF SOFTWARE CODING & INTEGRATION PROCESSES.....	48
TABLE FM.C-6	TESTING OF OUTPUTS OF INTEGRATION PROCESS.....	49
TABLE FM.C-7	VERIFICATION OF VERIFICATION PROCESS RESULTS	50
TABLE FM.C-8	SOFTWARE CONFIGURATION MANAGEMENT PROCESS.....	51
TABLE FM.C-9	SOFTWARE QUALITY ASSURANCE PROCESS.....	52
TABLE FM.C-10	SOFTWARE APPROVAL PROCESS	53
TABLE FM.B.1-1	REDUNDANCY MANAGEMENT SOFTWARE VERIFICATION RESULTS ..	B-7
TABLE FM.B.1-2	WINDOW MANAGER REQUIREMENTS FORMALIZED IN CTL.....	B-23
TABLE FM.B.1-3	WINDOW MANAGER ANALYSIS DATA	B-26

This Page Intentionally Left Blank

FM.1.0**INTRODUCTION**

Formal methods are mathematically based techniques for the specification, development, and verification of software aspects of digital systems. The mathematical basis of formal methods consists of formal logic, discrete mathematics, and computer-readable languages. The use of formal methods is motivated by the expectation that, as in other engineering disciplines, performing appropriate mathematical analyses can contribute to establishing the correctness and robustness of a design. For example, formal methods, because of their mathematical basis, are capable of:

- Unambiguously describing requirements of software systems.
- Enabling precise communication between engineers.
- Providing verification evidence such as consistency and accuracy of a formally specified representation of software.
- Providing verification evidence of the compliance of one formally specified representation with another.

Formal methods are also capable of demonstrating properties of software systems such as:

- Freedom from exceptions.
- Freedom from deadlock.
- Non-interference between different levels of criticality.
- Worst case execution time.
- Bounds on stack size during execution.
- Freedom from unintended function.
- Correct synchronous or asynchronous behavior.

Since the publication of DO-178B, advances and experience have been gained in formal methods, their application, and tools. This supplement provides guidance for applicants and certification or approval authorities to facilitate the use of formal methods.

The adoption of formal methods into an established set of processes for development and verification can be an evolutionary refinement rather than an abrupt change of methodology. The extent to which formal methods are used to satisfy the objectives of DO-178C can vary according to aspects such as preferences of the program management, choice of techniques, and availability of specialized resources. Formal methods might be used in a very selective manner to partially address a small set of objectives, or might be the primary source of evidence for the satisfaction of many of the objectives concerned with development and verification.

FM.1.1 Purpose

This supplement identifies the modifications and additions to DO-178C objectives, activities, explanatory text, and software life cycle data that should be addressed when formal methods are used as part of the software life cycle. This includes the artifacts that would be expressed using some formal notation and the verification evidence that could be derived from them.

FM.1.2 Scope

This supplement discusses the use of formal methods in the software life cycle for software that is produced in accordance with DO-178C. If the applicant is not planning to use formal analysis as defined in FM.1.6, then the applicant does not need to comply with this supplement.

FM.1.3 Relationship to Other Documents

This document supplements the guidance given in DO-178C and DO-278A. This document may be used in conjunction with any DO-178C-related supplements or other documents.

FM.1.4 How to Use This Document

This supplement should be used with and in the same way as DO-178C; however, the following should be noted:

- Section FM.1 contains explanatory text to aid the reader in understanding formal methods, and therefore is not to be taken as guidance.
- Annex FM.A of this supplement describes how the DO-178C objectives are revised in line with this formal methods guidance.
- Guidance from DO-178C should be used for all aspects of the software life cycle, apart from where formal methods are used. All guidance specific to formal methods is contained within this supplement.
- This supplement can be applied to DO-278A in the same manner as DO-178C. Applicants complying with DO-278A should be aware that the text within this supplement is based on DO-178C; thus, the DO-278A applicant needs to ensure that the proper DO-278A guidance is addressed. Annex FM.B of this supplement describes how the DO-278A, Annex A objectives are revised in line with the formal methods guidance in this supplement.
- In addition to the guidance contained in this supplement, a Discussion Paper is provided to clarify the guidance (see Appendix FM.B). The Discussion Paper contains no new or additional guidance.
- Major sections are numbered as FM.X.0 throughout this supplement. It should be noted that references to an entire section are identified as “section FM.X”; whereas, references to the content between section headers FM.X.0 and FM.X.1 are referred to as “section FM.X.0”.

FM.1.5 Document Overview

This supplement is intended to be used with DO-178C. Sections FM.4, FM.5, FM.6, FM.11, FM.12, and Annex FM.A provide additional and modified text to the referenced sections of DO-178C. This supplement minimizes any redundancy between DO-178C and the document itself. Each affected section of DO-178C is included in this supplement. Where the guidance from DO-178C applies unchanged, this is stated in this supplement. Text taken directly from DO-178C is shown in italics.

In Annex FM.A, only those tables with changes are included. These tables include new objectives and changes to referenced text for existing objectives.

FM.1.6 Characteristics of Formal Methods

Understanding some of the fundamental characteristics of formal methods and their scope of applicability is essential to appropriate application of the guidance in this supplement. Application of formal methods usually involves two types of activities: modeling and analysis, where the models and analysis procedures are derived from or defined by an underlying mathematically precise foundation. Although modeling and analysis are common activities in many software engineering efforts, these terms have particular meanings when used in the context of formal methods described in this supplement. Not all models or all analyses will meet requirements for being formal. Throughout this supplement, the terms formal models and formal analysis are used to distinguish those activities when they meet requirements specified herein for formality. A formal model combined with a formal analysis constitutes a formal method. If the applicant plans to use formal models but not formal analyses, then they do not need to comply with this supplement.

FM.1.6.1 Formal Models

Establishing a formal model of the software artifact of interest is fundamental to all formal methods. In general a model is an abstract representation of a given set of aspects of the software that is used for analysis, simulation, and/or code generation. In the context of this supplement, to be formal, a model should have an unambiguous, mathematically defined syntax and semantics. This makes it possible to use automated means to obtain guarantees that the model has certain specified properties.

Formal models can be produced by the development process (for example, requirement or design activities) or by formal analysis of the software artifact. Creating a formal model by formal analysis of a software artifact is possible when the artifact itself exhibits the characteristics of well-defined syntax and semantics, such as Source Code and Executable Object Code.

Throughout the development life cycle, different formal models may be used for different kinds of analysis and for the establishment of different properties. Examples of formal models include:

- Graphical models where the components of the diagram and the connections between them have mathematically defined syntax and semantics. For example, including but not limited to control diagrams or state machines.
- Textual based models that have a mathematically defined syntax and semantics. Example could be: Z, logic, set theory, algebraic or differential equations, subsets of C, subsets of Ada or other languages.

- Abstract models formed inside a formal methods tool by an analysis of the software artifacts of interest such as the code.

The applicability of formal methods to any particular software development activity is bounded by the ability to construct an appropriate formal model. A single model, or even a combination of models, may not be sufficient to capture all of the aspects of the software that are required to be verified according to the objectives of DO-178C. That is, in many cases, it is not possible to completely model the software using a formal notation. Requirements specified in natural language (such as English) may include properties that cannot be verified with a formal method. Models can also be insufficiently detailed to allow meaningful analysis of some properties and yet be perfectly adequate for others.

Despite these limitations, formal models can be a means of completely and accurately describing important software properties. Formal analyses can then be applied to provide assurance of these properties, and combined with other verification techniques, (for example, testing) to achieve satisfaction of all of the verification objectives. Formal methods provide comprehensive assurance of particular properties only for those aspects that are formalized in the formal model, so defining the limits of the model is essential.

Formal methods may be applied in many stages of the development life cycle, and to satisfy various verification objectives. Wherever formal models are used, they should be verified to provide assurance that they are an accurate representation of the artifact. This means that for the properties being analyzed, if they hold for the model then they hold for the actual artifact.

Note: One method for ensuring the correct development of models is the use of approved DO-178C guidance for model-based development.

FM.1.6.2 Formal Analysis

Although there are important benefits in creating formal models of life cycle artifacts, the most powerful benefits of formal methods are in the formal analysis of those models. Formal analysis can provide guarantees or proofs of software properties and compliance with requirements. Proof, or guarantee, implies that all execution cases are taken into account, achieving exhaustive verification. To conduct a formal analysis, a set of software properties is necessary. These properties are either created or embedded in specific tools that implement the formal analysis (for example, worst-case execution timing). Properties for formal analysis can be seen as analogous in many ways to verifiable requirements for testing. In testing, test cases should be easily derivable from verifiable requirements; for formal analysis, the properties are usually also the verification cases. It may be possible to define a complete set of properties for a set of requirements. In this case, proof that the set of properties always holds true demonstrates the equivalence of the item under analysis to its requirements.

For the purpose of this supplement, an analysis method can only be regarded as formal analysis if its determination of a property is sound. Sound analysis means that the method never asserts a property to be true when it is not true. The converse case, the assertion that a property is false when it may be true, colloquially “the raising of false alarms”, is a usability issue but not a soundness issue. Furthermore, it is acceptable for a method to return “don’t know” or not to return an answer when trying to establish whether a property holds, in which case additional verification is necessary.

There are many different kinds of formal analysis, but they can typically be classified in three categories: (1) deductive methods, such as theorem proving, (2) model checking, and (3) abstract interpretation.

- (1) Deductive methods involve mathematical arguments, such as mathematical proofs, for establishing a specified property of a formal model. A correct proof of a property provides rigorous evidence of that property for the formal model. These proofs are typically constructed using an automated or interactive theorem proving system. Even with such assistance, constructing proofs can be difficult, or impossible in some cases. However, once a proof is completed, automatic checking of the correctness of that proof is usually trivial.
- (2) Model checking explores all possible behaviors of a formal model to determine whether a specified property is satisfied. In cases where the property is not satisfied, a counter-example illustrating where and how the property fails to hold is generated automatically. In some cases, a model checker may not be able to determine whether the given property is satisfied or not.
- (3) Abstract interpretation is a theory for formally constructing conservative representations (that is, enforcing soundness) of the semantics of programming languages. In practice, this method is used for constructing semantics-based analysis algorithms for the automatic, static, and sound determination of dynamic properties of infinite-state programs. With abstract interpretation, a formal model is generated, usually within a tool, which is specific to the particular property under analysis. It can be viewed as a partial execution of a computer program which determines specific effects of the program (for example, control structure, flow of information, stack size, number of clock cycles) without actually performing all the calculations.

All of these formal analyses share two important attributes. First, the methods rely on formal models. Compliance between artifacts can never be demonstrated between an informal model and a formal model, using formal analysis. For example, using formal methods to demonstrate compliance between specification and code requires that both be formal models. Second, all of the formal analyses are generally implemented using a tool. Any tool that supports the formal analysis should be assessed under the tool qualification guidance required by DO-178C and qualified where necessary.

Scalability of formal analysis can be an important issue because formal analysis examines all possible behavior of the formal model. As the size of the model grows, the computing power required to analyze the model may become exponentially greater.

FM.1.7

Formal Methods as a Verification Technique

Formal methods may enable complete exploration of possible behaviors of a formal model with respect to certain properties. Subject to the formal model employed being an accurate representation of the artifact (which should be established by additional verification, as per guidance in this supplement), formal methods provide assurance that certain kinds of faults are not present at the level of description and stage of the life cycle considered by the formal model. This supplement provides guidance for using this assurance to meet verification objectives of DO-178C. Some objectives may be fully met using formal methods. Other objectives may require additional verification to supplement formal methods, such as complementary testing in the integrated target computer environment.

This Page Intentionally Left Blank

FM.2.0 SYSTEM ASPECTS RELATING TO SOFTWARE DEVELOPMENT

Section 2 of DO-178C is unchanged.

This Page Intentionally Left Blank

FM.3.0 SOFTWARE LIFE CYCLE

Section 3 of DO-178C is unchanged.

This Page Intentionally Left Blank

FM.4.0 SOFTWARE PLANNING PROCESS

DO-178C sections 4.1, 4.2, and 4.4 to 4.6 are unchanged. Some life cycle data items, referred to in section 4.5 of DO-178C, have been modified to address formal notations.

The applicant should ensure that the intended use of this supplement is acceptable to the appropriate certification authority. Several supplements may be applied to an applicant's software life cycle. The applicant should plan an approach that will comply with the objectives of DO-178C and those applicable objectives from the supplements. As such, all relevant supplements and the supplements' associated objectives should be considered. If the applicant's particular approach identifies a conflict between the objectives, then the applicant should propose a solution to the conflict in the Plan for Software Aspects of Certification (PSAC). The applicant may use a single set of software plans or multiple set of software plans addressing the different supplement. Using a single set of software plans, containing sections addressing each supplement, will provide an integrated approach with a clear and comprehensive plan.

As supplements only extend guidance from DO-178C for a specific technology, the applicant should first consider all of the DO-178C objectives, and then the supplement's objectives, and lastly any other additional considerations. One approach is to consolidate all objectives of DO-178C and the applicable supplements, and for each objective provide a statement of how compliance will be achieved, along with identifying any applicable life cycle data items. Because the objective numbering scheme is unique between the Annex A Tables of DO-178C and the Annex A Tables of the supplements, the identification of the objectives and their document sources should be clear and unambiguous.

FM.4.3 Software Plans

The software plans define the means of satisfying the objectives of DO-178C and other applicable documents. They specify the organizations that will perform those activities. The software plans are:

- *The Plan for Software Aspects of Certification (DO-178C section 11.1) serves as the primary means for communicating the proposed development methods to the certification authority for agreement, and defines the means of compliance. The Plan for Software Aspects of Certification gives an overview of how and where formal methods will be used and what evidence those methods will provide.*
- *The Software Development Plan (DO-178C section 11.2) defines the software life cycle(s), and software development environment, and the means by which the software development process objectives will be satisfied. The Software Development Plan details the precise development steps that will include the use of formal methods.*
- *The Software Verification Plan (DO-178C section 11.3) defines the means by which the software verification process objectives will be satisfied. The Software Verification Plan details the methods used to gather the verification evidence including that obtained by formal analysis. All assumptions related to the formal analysis should be described and justified, especially those associated with the target computer. The effects of these assumptions on the ability to detect errors and verify functionality using formal analysis should be considered, and additional software verification processes and methods should be specified in the Software Verification*

Plan. Consideration should be given as to how the various verification techniques, such as testing and formal analysis, are combined to meet all the verification objectives for the software. Specifically the overlap or potential gaps between the methods need to be resolved, and may affect the integration test planning. This is necessary as formal methods can be applied to partially meet objectives or fully meet objectives for a subset of the software. They can also be used to verify all of the software for specific objectives.

- There is no additional guidance for formal methods related to the Software Configuration Management and Software Quality Assurance Plans (DO-178C sections 11.4 and 11.5).

The remainder of DO-178C section 4.3 is unchanged.

FM.5.0**SOFTWARE DEVELOPMENT PROCESSES**

This section discusses the objectives and activities of the software development processes. The software development processes are applied as defined by the software planning process (DO-178C section 4) and the Software Development Plan (DO-178C section 11.2). DO-178C Table A-2 of Annex A is a summary of the objectives and outputs of the software development processes by software level. The software development processes are:

- *Software requirements process.*
- *Software design process.*
- *Software coding process.*
- *Integration process.*

Software development processes produce one or more levels of software requirements. High-level requirements are produced directly through analysis of system requirements and system architecture. Usually, these high-level requirements are further developed during the software design process, thus producing one or more successive, lower levels of requirements. However, if Source Code is generated directly from high-level requirements, then the high-level requirements are also considered low-level requirements and the guidance for low-level requirements also apply.

Note: The applicant may be required to justify software development processes that produce a single level of requirements.

The development of software architecture involves decisions made about the structure of the software. During the software design process, the software architecture is defined and low-level requirements are developed. Low-level requirements are software requirements from which Source Code can be directly implemented without further information.

Each software development process may produce derived requirements. Some examples of requirements that might be determined to be derived requirements are:

- *The need for interrupt handling software to be developed for the chosen target computer.*
- *The specification of a periodic monitor's iteration rate when not specified by the system requirements allocated to software.*
- *The addition of scaling limits when using fixed point arithmetic.*

High-level requirements may include derived requirements, and low-level requirements may include derived requirements. In order to determine the effects of derived requirements on the system safety assessment and system requirements, all derived requirements should be made available to the system processes including the system safety assessment process.

Using formal models to define requirements or design artifacts allows some of the verification objectives to be satisfied by the use of formal analysis.

Formalizing requirements and/or design may increase the effort required to specify them, but, generally, additional errors are found and removed earlier during this process due to the additional scrutiny and standardization.

Not all of the requirements need to be defined formally when using a formal method. The key to using a formal method is that there is sufficient definition in the formal model to verify properties about the specific artifacts being analyzed. For example, if a formal model is created at the architecture level, then the software architecture can be analyzed independently of low-level requirements. In the same way, if a formal model is created at the low-level requirements level, then low-level requirements can be analyzed independently of the high-level requirements and software architecture. Some methods create a formal model at the source code level, embedding information flow from the design into the code, which is then checked by formal analysis. It may even be possible to apply formal methods to analyze properties of object code. In this case, object code becomes a formal model, the semantics of which are treated the same by the formal analysis as they are by the target hardware.

DO-178C sections 5.1 to 5.5 are unchanged.

FM.6.0 SOFTWARE VERIFICATION PROCESS

This section discusses the objectives and activities of the software verification process. Verification is a technical assessment of the outputs of the software planning process, software development processes, and the software verification process. The software verification process is applied as defined by the software planning process (DO-178C section 4) and the Software Verification Plan (DO-178C section 11.3). See DO-178C section 4.6 for the verification of the outputs of the planning process.

Verification is not simply testing. Testing, in general, cannot show the absence of errors. As a result, the following sections use the term "verify" instead of "test" to discuss the software verification process activities, which are typically a combination of reviews, analyses, and tests. Guidance for reviews, analyses (other than formal), and testing is given in DO-178C.

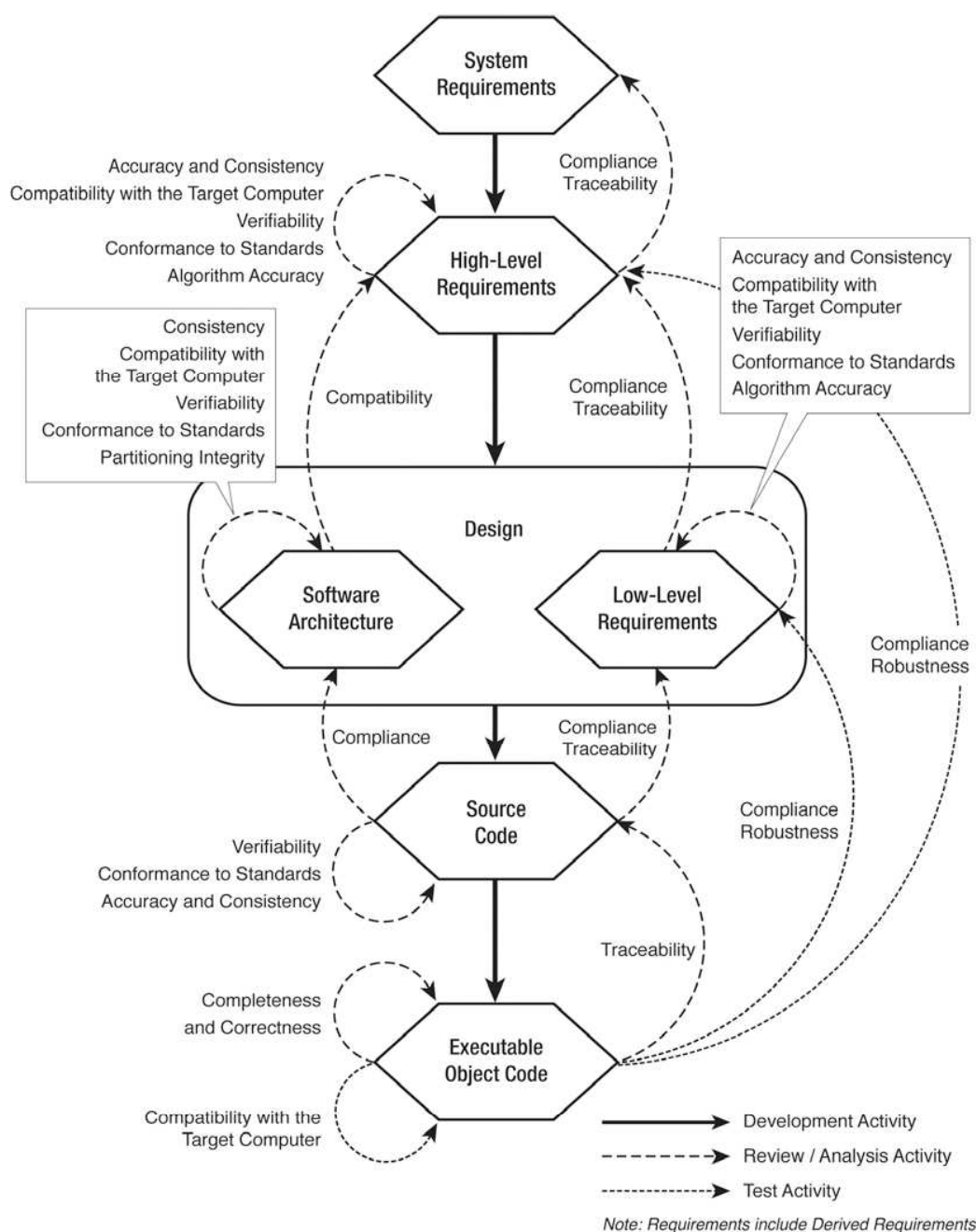
With formal analyses, the correctness of life cycle data with respect to a formal model can be proved or disproved. The extent to which formal analysis is used to satisfy the verification objectives can vary according to the ability to construct appropriate formal models, the choice of analysis techniques, and the availability of tools and related resources. Formal analysis might be used to partially address a small set of the verification objectives, or it might be the primary source of evidence for the satisfaction of most of the objectives concerned with verification. As a minimum, testing will still be required to ensure that the Executable Object Code is compatible with the target computer.

FM.6.1 Purpose of Software Verification

The purpose of the software verification process as described in DO-178C section 6.1 is unchanged. To help explain where formal analysis fits in the software verification process as described in the following sections (FM.6.2–FM.6.6), it is helpful to look at the specific verification objectives. Figure FM.6-1 shows the relationships for Level A among life cycle data items, development activities, and verification activities with respect to objectives in DO-178C Tables A-3 through A-6. Each verification activity is depicted by a dashed line and the objectives which it satisfies are annotated on the arrows. The life cycle data items are shown as hexagons. Solid lines are used to signify development activities.

Note: Not all life cycle data items involved in verification are shown on the diagram. For example, standards are taken as implicit for activities such as “Conformance to standards.” Verification of outputs of software testing as defined in DO-178C Table A-7 is also not shown.

As shown in Figure FM.6-1, some of the verification objectives appear repeatedly. For example, there are objectives for traceability and compliance between the high-level requirements and systems requirements, between the low-level requirements and high-level requirements, and between Source Code and low-level requirements. Formal analysis can be used to satisfy many of the verification objectives, completely in some cases and only partly in others. In this last case, the verification plan should describe how the combination of formal analysis and other methods satisfies the objective completely.



**Figure FM.6-1 Level A Software Verification Processes
(DO-178C TABLES A-3 TO A-6)**

FM.6.2 Overview of Software Verification Process Activities

DO-178C section 6.2 is unchanged, except for the addition of section FM.6.2.1.

FM.6.2.1 Considerations for Formal Methods

When formal analysis is used to meet a verification objective, there is an additional objective to ensure that each formal method used is correctly defined, justified, and appropriate to meet this verification objective. Activities for this are as follows:

- a. All notations used for formal analysis should be verified to have precise, unambiguous, mathematically defined syntax and semantics; that is, they are formal notations.
- b. The soundness of each formal analysis method should be justified. A sound method never asserts that a property is true when it may not be true.
- c. All assumptions related to each formal analysis should be described and justified; for example, assumptions associated with the target computer or about the data range limits.

FM.6.3 Software Reviews and Analyses

Reviews and analyses are applied to the outputs of the software development processes. One distinction between reviews and analyses is that analyses provide repeatable evidence of correctness and reviews provide a qualitative assessment of correctness. A review may consist of an inspection of an output of a process guided by a checklist or similar aid. An analysis may examine in detail the functionality, performance, traceability, and safety implications of a software component, and its relationship to other components within the system or equipment.

Formal analysis can be used to satisfy the objectives for software reviews and analysis as follows:

- a. Compliance: If the input life cycle data items and the output life cycle data items of a software development process are formally modeled, then formal analysis can be used to verify compliance. However, formal methods cannot show that a derived requirement is correctly defined and has a reason for existing; this should be achieved by review.
- b. Accuracy: Formal notations are precise and unambiguous, and can be used to demonstrate accuracy of the representation of a life cycle data item.
- c. Consistency: Life cycle data items that are formally expressed can be checked for consistency (the absence of conflicts). If a set of formal statements is found to be logically inconsistent, then the inconsistencies should be addressed before any subsequent analysis is conducted.
- d. Compatibility with the target computer: Formal analysis can be used to detect potential conflicts between a formal description of the target computer and the life cycle data item.

- e. Verifiability: Being able to express a requirement in the formal notation defined in the software verification plan is evidence of verifiability because it is unambiguous and has a mathematically defined syntax and semantics and then could be formally analyzed.

Note: Requirements involving “always/never” cannot generally be verified by a finite set of test cases, but may be verified by formal analysis.

- f. Conformance to standards: Life cycle data items that are formally expressed should be compliant with any standards defining the formalism.

Note: Invalid results will be obtained if ill-formed requirements are allowed. Since formal notations have a precise, unambiguous, and mathematically defined syntax, automated syntax checkers are appropriate for verifying that the formally stated requirements are well-formed with respect to syntax. In addition, an automated checker may enforce other restrictions on the notation (for example, complexity of notational constructs and other design constructs that would not comply with the system safety objectives). Automated checking will need to be supplemented by review for those standards not amenable to automated checking.

- g. Traceability: Traceability ensures that all input requirements have been developed into lower-level requirements or Source Code. Traceability from the inputs of a process to its outputs can be demonstrated by verifying with a formal analysis that the outputs of the process fully satisfy its inputs. Traceability from the outputs of a process to its inputs can be demonstrated by verifying with a formal analysis that each output data item is necessary to satisfy some input data item.

- h. Algorithm aspects: If life cycle data items are formally modeled, then algorithmic aspects can be checked using formal analysis.

- i. Requirement formalization correctness: If a requirement has been translated to a formal notation as the basis for using a formal analysis, then review or analysis should be used to demonstrate that the formal statement is a conservative representation of the informal requirement.

Note: If the gap between an informal statement of the requirement and its embodiment in a formal notation is too large, then this may be difficult to review. The preciseness of formal notations is only an advantage when they maintain fidelity to the intent of the informal requirement.

FM.6.3.1 Reviews and Analyses of High-Level Requirements

These review and analysis activities detect and report requirements errors that may have been introduced during the software requirements process. These verification activities confirm that the high-level requirements satisfy the objectives that follow. A mixture of both formal and informal requirements can be used for either or both system and high-level requirements. Formal methods can be used to meet all or part of the following objectives:

- a. *Compliance with system requirements: The objective is to ensure that the system functions to be performed by the software are defined, that the functional, performance, and safety-related requirements of the system are satisfied by the high-level requirements, and that derived requirements and the reason for their existence*

are correctly defined. If the system requirements and high-level requirements are formally modeled, then formal analysis can be used to show compliance. Formal methods cannot show that derived requirements and the reason for their existence are correctly defined; this should be achieved by review.

- b. *Accuracy and consistency:* *The objective is to ensure that each high-level requirement is accurate, unambiguous, and sufficiently detailed, and that the requirements do not conflict with each other.* If the high-level requirements are expressed in a formal notation, then they will be precise and unambiguous. The formal model of high-level requirements can be checked for consistency (the absence of conflicts) and may enable accuracy checks to be carried out using formal analysis.
- c. *Compatibility with the target computer:* *The objective is to ensure that no conflicts exist between the high-level requirements and the hardware/software features of the target computer, especially system response times and input/output hardware.* If the high-level requirements and hardware/software features of the target computer are formally modeled, then potential conflicts can be detected through formal analysis.
- d. *Verifiability:* *The objective is to ensure that each high-level requirement can be verified.* Being able to express a high-level requirement in the formal notation defined in the Software Verification Plan is evidence of verifiability.
- e. *Conformance to standards:* *The objective is to ensure that the Software Requirements Standards were followed during the software requirements process and that deviations from the standards are justified.* High-level requirements expressed in a formal notation can be checked for conformance to standards by using formal analysis.
- f. *Traceability:* *The objective is to ensure that the functional, performance, and safety-related requirements of the system that are allocated to software were developed into the high-level requirements.* If formal methods are used to meet the objective of section FM.6.3.1, item a, this can provide evidence in support of traceability.
- g. *Algorithm aspects:* *The objective is to ensure the accuracy and behavior of the proposed algorithms, especially in the area of discontinuities.* If high-level requirements are formally modeled, then they can be checked for accuracy and behavior by using formal analysis.

FM.6.3.2 Reviews and Analyses of Low-Level Requirements

These review and analysis activities detect and report requirements errors that may have been introduced during the software design process. Reviews and analyses may be used to confirm that the software low-level requirements satisfy the objectives below. A mixture of both formal and informal requirements can be used for either or both high-level and low-level requirements. Formal methods can be used to meet all or part of the following objectives:

- a. *Compliance with high-level requirements:* *The objective is to ensure that the low-level requirements satisfy the high-level requirements and that derived requirements and the design basis for their existence are correctly defined.* If the high-level requirements and low-level requirements are formally modeled, then formal analysis can be used to show compliance. Formal methods cannot show that derived requirements and the reason for their existence are correctly defined; this should be achieved by review.

- b. Accuracy and consistency: The objective is to ensure that each low-level requirement is accurate and unambiguous, and that the low-level requirements do not conflict with each other. If the low-level requirements are expressed in a formal notation, then they will be precise and unambiguous. The formal model of low-level requirements can be checked for consistency (the absence of conflicts) and may enable accuracy checks to be carried out using formal analysis.
- c. Compatibility with the target computer: The objective is to ensure that no conflicts exist between the low-level requirements and the hardware/software features of the target computer, especially the use of resources such as bus loading, system response times, and input/output hardware. If the low-level requirements and hardware/software features of the target computer are formally modeled, then potential conflicts can be detected through formal analysis.
- d. Verifiability: The objective is to ensure that each low-level requirement can be verified. Being able to express a low-level requirement in the formal notation defined in the Software Verification Plan is evidence of verifiability.
- e. Conformance to standards: The objective is to ensure that the Software Design Standards were followed during the software design process and that deviations from the standards are justified. Low-level requirements expressed in a formal notation can be checked for conformance to standards by using formal analysis.
- f. Traceability: The objective is to ensure that the high-level requirements and derived requirements were developed into the low-level requirements. If formal methods are used to meet the objective of section FM.6.3.2, item a, this can provide evidence in support of traceability.
- g. Algorithm aspects: The objective is to ensure the accuracy and behavior of the proposed algorithms, especially in the area of discontinuities. If low-level requirements are formally modeled, then they can be checked for accuracy and behavior by using formal analysis.

FM.6.3.3 Reviews and Analyses of Software Architecture

These review and analysis activities detect and report errors that may have been introduced during the development of the software architecture. Reviews and analyses may be used to confirm that the software architecture satisfies the objectives below. A mixture of both formal and informal requirements can be used for either or both high-level requirements and software architecture. Formal methods can be used to meet all or part of the following objectives:

- a. Compatibility with the high-level requirements: The objective is to ensure that the software architecture does not conflict with the high-level requirements, especially functions that ensure system integrity, for example, partitioning schemes. If the high-level requirements and software architecture are formally modeled, then formal analysis can be used to show compatibility.
- b. Consistency: The objective is to ensure that a correct relationship exists between the components of the software architecture. This relationship exists via data flow and control flow. If the interface is to a component of a lower software level, it should also be confirmed that the higher software level component has appropriate protection mechanisms in place to protect itself from potential erroneous inputs from the lower software level component. If the software architecture is expressed in a

formal notation, then it will be precise and unambiguous. Data and control flow between components can be checked for correct relationships using formal analysis.

- c. Compatibility with the target computer: The objective is to ensure that no conflicts exist, especially initialization, asynchronous operation, synchronization, and interrupts, between the software architecture and the hardware/software features of the target computer. If the software architecture and hardware/software features of the target computer are formally modeled, then potential conflicts can be detected through formal analysis.
- d. Verifiability: The objective is to ensure that the software architecture can be verified, for example, there are no unbounded recursive algorithms. Being able to express the software architecture in the formal notation as defined in the Software Verification Plan is evidence of verifiability.
- e. Conformance to standards: The objective is to ensure that the Software Design Standards were followed during the software design process and that deviations to the standards are justified, for example, deviations to complexity restrictions and design construct rules. Software architecture expressed in a formal notation can be checked for conformance to standards by using formal analysis.
- f. Partitioning integrity: The objective is to ensure that partitioning breaches are prevented. If the software architecture is formally modeled, some aspects of partitioning integrity can be verified by formal analysis.

FM.6.3.4 Reviews and Analyses of Source Code

These review and analysis activities detect and report errors that may have been introduced during the software coding process. Reviews and analyses can be used to confirm that the outputs of the software coding process are accurate, complete and can be verified. *Primary concerns include correctness of the code with respect to the software requirements and the software architecture, and conformance to the Software Code Standards.* If formal source code semantics exist, compliance with some or all of the following objectives can be achieved through formal methods.

- a. Compliance with the low-level requirements: The objective is to ensure that the Source Code is accurate and complete with respect to the low-level requirements and that no Source Code implements an undocumented function. If the low-level requirements are formally modeled and formal source code semantics exist, then formal analysis can be used to show compliance.
- b. Compliance with the software architecture: The objective is to ensure that the Source Code matches the data flow and control flow defined in the software architecture. If the software architecture is formally modeled and formal source code semantics exist, then data and control flow can be analyzed using formal analysis.
- c. Verifiability: The objective is to ensure the Source Code does not contain statements and structures that cannot be verified and that the code does not have to be altered to verify it. When Source Code has mathematically defined syntax and semantics and is consistent with the formal analysis defined in the Software Verification Plan, then this objective is satisfied.
- d. Conformance to standards: The objective is to ensure that the Software Code Standards were followed during the development of the code, for example, complexity

restrictions and code constraints. Complexity includes the degree of coupling between software components, the nesting levels for control structures, and the complexity of logical or numeric expressions. This analysis also ensures that deviations to the standards are justified. Source Code with mathematically defined syntax and semantics can be checked for conformance to standards by using formal analysis.

- e. Traceability: *The objective is to ensure that the low-level requirements were developed into Source Code.* When formal methods are used to meet the objective of section FM.6.3.4, item a, this can provide evidence in support of traceability.
- f. Accuracy and consistency: *The objective is to determine the correctness and consistency of the Source Code, including stack usage, memory usage, fixed point arithmetic overflow and resolution, floating-point arithmetic, resource contention and limitations, worst-case execution timing, exception handling, use of uninitialized variables, cache management, unused variables, and data corruption due to task or interrupt conflicts. The compiler (including its options), the linker (including its options), and some hardware features may have an impact on the worst-case execution timing and this impact should be assessed.* If mathematically defined syntax and semantics exist for the Source Code, then these characteristics can be checked using formal analysis. The mathematically defined syntax and semantics may need to take into account the programming language standards, compiler information (for example, default behavior and configuration options), and characteristics of the target computer.

FM.6.3.5 Reviews and Analyses of the Outputs of the Integration Process

DO-178C section 6.3.5 is unchanged.

FM.6.3.6 Reviews and Analyses of the Formal Analysis Cases, Procedures, and Results for Requirements, Architecture, and Source Code

When formal analysis is used to meet a verification objective, it is necessary to ensure that the formal analysis was developed and performed accurately and completely for that objective. The topics should include:

- a. Formal analysis cases: The objective is to provide evidence that the formal analysis covers the objectives in sections FM.6.3.1–FM.6.3.4. All assumptions that were included in the formal analysis should be justified and any false assumptions (which would invalidate the analysis) should be identified and addressed.
- b. Formal analysis procedures: The objective is to verify that the formal analysis cases were accurately developed into formal analysis procedures and expected results.
- c. Formal analysis results: The objective is to ensure that the formal analysis results are correct and that discrepancies between actual and expected results are explained.

FM.6.4 Software Testing

If formal methods are being used to verify Executable Object Code, then FM.6.7 should be used. DO-178C section 6.4 applies if formal methods are not being used to verify Executable Object Code.

Note: The text of FM.6.7 is derived from DO-178C section 6.4, with modification for formal analysis. The text in FM.6.7 is not italicized because there is no corresponding section in DO-178C.

FM.6.5 Software Verification Process Traceability

Software verification process traceability activities include:

- a. Trace Data, showing the bi-directional association between software requirements and verification cases, is developed. This Trace Data supports the requirements-based coverage analysis.*
- b. Trace Data, showing the bi-directional association between verification cases and verification procedures, is developed. This Trace Data enables verification that the complete set of verification cases has been developed into verification procedures.*
- c. Trace Data, showing the bi-directional association between verification procedures and verification results, is developed. This Trace Data enables verification that the complete set of verification procedures has been executed.*

FM.6.6 Verification of Parameter Data Items

DO-178C section 6.6 is unchanged.

FM.6.7 Formal Analysis of the Executable Object Code

Verification of the Executable Object Code is primarily performed by testing. This can be assisted by reviews and analyses as indicated in DO-178C. Formal analyses exist to establish specific properties such as worst-case execution time and stack usage. These analyses can replace some testing aspects of the Executable Object Code, but not all of them. Tests executed in target hardware are always required to ensure that the software in the target computer will satisfy the high-level requirements as defined in DO-178C section 6.4.3, item a.

Verification activities for the Executable Object Code are used to demonstrate that the software satisfies its requirements and to demonstrate with a high degree of confidence that errors which could lead to unacceptable failure conditions, as determined by the system safety assessment process, have been removed.

The verification activities should verify:

- Correct operation of the software in the target computer environment.
- Correct inter-relationships between software requirements and components and the correct implementation of the software requirements and software components within the software architecture. If a combination of formal methods and testing has been planned by the applicant, then verification of these interrelationships should be achieved by integration testing as stated in DO-178C.
- Correct implementation of software low-level requirements.

Note: The above list is in order of precedence; that is, you may be able to achieve all three by verification in the target computer environment.

When using formal methods:

- (1) Formal analysis cases should be based primarily on the software requirements.
- (2) Formal analysis cases should be developed to verify correct functionality and to establish conditions that reveal potential errors.
- (3) Software requirements coverage analysis should determine what software requirements were not verified.
- (4) Structural coverage analysis should determine what software structures were not verified.

Guidance for requirements-based formal analysis case selection includes:

- Normal range and robustness (abnormal range) cases should be considered.
- The specific cases should be developed from the software requirements and the error sources inherent in the software development processes.

The objectives for verification of the Executable Object Code include:

- a. Compliance with the high-level requirements: The objective is to demonstrate the ability of the software to respond to normal inputs and conditions. Activities for normal range cases include:
 - Real and integer input variables should be considered using valid boundary values.
 - For time-related functions, such as filters, integrators and delays, multiple iterations of the code should be performed to check the characteristics of the function in context.
 - For state transitions, analyses should be developed to consider the transitions possible during normal operation.
 - For software requirements expressed by logic equations, the normal range cases should verify the variable usage and the Boolean operators.

Formal methods can be used to satisfy this objective as described in FM.6.7.f.

- b. Robustness for high-level and low-level requirements: The objective is to demonstrate the ability of the software to respond to abnormal inputs and conditions. Activities for robustness cases include:
 - Real and integer variables should be considered.
 - System initialization should be considered during abnormal conditions.
 - The possible failure modes of the incoming data should be determined, especially complex, digital data strings from an external system.
 - For loops where the loop count is a computed value, formal analysis cases should be developed to attempt to compute out-of-range loop count values, and thus demonstrate the robustness of the loop-related code.

- A check should be made to ensure that protection mechanisms for exceeded frame times respond correctly.
- For time-related functions, such as filters, integrators and delays, formal analysis cases should be developed for arithmetic overflow protection mechanisms.
- For state transitions, formal analysis cases should be developed to provoke transitions that are not allowed by the software requirements.

Formal methods can be used to satisfy this objective as described in FM.6.7, item f.

- c. Verification of software integration: The verification activity should concentrate on the inter-relationships between the software requirements, and on the implementation of requirements by the software architecture. The objective is to ensure that the software components interact correctly with each other and satisfy the software requirements and software architecture. Typical errors to consider include:

- Incorrect initialization of variables and constants.
- Parameter passing errors.
- Data corruption, especially global data.
- Inadequate end-to-end numerical resolution.
- Incorrect sequencing of events and operations.

Formal methods can be used to verify the absence of some of the error types listed above, as described in FM.6.7, item f. Additionally, testing will be required to verify the software integration in target hardware.

- d. Compliance with the low-level requirements: The objective is to demonstrate the ability of the software to respond to normal inputs and conditions. Activities for normal range cases include:

- Real and integer input variables should be considered using valid boundary values.
- For time-related functions, such as filters, integrators and delays, multiple iterations of the code should be performed to check the characteristics of the function in context.
- For state transitions, analyses should be developed to consider the transitions possible during normal operation.
- For software requirements expressed by logic equations, the normal range cases should verify the variable usage and the Boolean operators.

Formal methods can be used to satisfy this objective as described in FM.6.7, item f.

Typical errors to consider include:

- Failure of an algorithm to satisfy a software requirement.
- Incorrect loop operations.

- Incorrect logic decisions.
 - Failure to process correctly legitimate combinations of input conditions.
 - Incorrect responses to missing or corrupted input data.
 - Incorrect handling of exceptions, such as arithmetic faults or violations of array limits.
 - Incorrect computation sequence.
 - Inadequate algorithm precision, accuracy or performance.
- e. Compatibility with the target computer: This verification activity should concentrate on error sources associated with the software operating within the target computer environment, and on the high-level functionality. The objective is to ensure that the software in the target computer will satisfy the high-level requirements. This is achieved primarily by testing. Typical errors to consider include:
- Incorrect interrupt handling.
 - Failure to satisfy execution time requirements.
 - Incorrect software response to hardware transients or hardware failures, for example, start-up sequencing, transient input loads and input power transients.
 - Data bus and other resource contention problems, for example, memory mapping.
 - Inability of built-in test to detect failures.
 - Errors in hardware/software interfaces.
 - Incorrect behavior of control loops.
 - Incorrect control of memory management hardware or other hardware devices under software control.
 - Stack overflow.
 - Incorrect operation of mechanism(s) used to confirm the correctness and compatibility of field-loadable software.
 - Violations of software partitioning.

Formal methods can be used to verify the absence of some of the errors listed above providing that all of the following are satisfied:

- (1) A formal model of the Executable Object Code with respect to the error condition is defined. This model should be complete with respect to the necessary hardware characteristics for the specific error type (for example, as long as all types of usage of the stack and instances of usage of the stack are modeled and the available stack size is defined, then a stack overflow check is achievable).

- (2) Formal evidence demonstrates the absence of some of the errors listed above in the Executable Object Code, either by computing safe upper bounds of resource consumption (for example stack usage, worst-case execution timing) or by proving formally expressed properties (for example absence of resource contention).

f. Verification of Property Preservation Between Source and Executable Object Code

Options for verification activities of Executable Object Code include:

- (1) Formal analysis of Executable Object Code can be used to satisfy the objectives if all of the following conditions are satisfied:
 - The requirements are formally expressed.
 - A formal model of the Executable Object Code is defined.
 - Formal evidence demonstrates that the formal model of the Executable Object Code satisfies the requirements.
- (2) Formal analysis of Source Code can be used to satisfy the objectives if all of the following conditions are satisfied:
 - The requirements are formally expressed.
 - A formal model of the Source Code is defined.
 - Formal evidence demonstrates that the formal model of the Source Code satisfies the requirements.
 - Complementary analysis shows property preservation between the Source Code and the EOC. By verifying the correctness of the translation of source to object code, formal analysis performed at the Source Code level against high or low-level requirements can be used to infer correctness of the Executable Object Code against high or low-level requirements. This is similar to the way that coverage metrics gained from Source Code can be used to establish the adequacy of tests to verify the target system. If a compiler, linker, or other means generates additional code that is not directly traceable to Source Code statements, then additional verification should be performed to establish property preservation.

Figure FM.6-2 shows that where formal methods are used at source level to provide evidence of the compliance and robustness of the Executable Object Code to the high level or low-level requirements, the verification of property preservation between Source Code and Executable Object Code should be demonstrated. The diagram shows alternative verification paths; for example, formal methods could show compliance between Executable Object Code and low-level requirements (via property preservation in the object code), instead of testing, and testing could show compliance between Executable Object Code and high-level requirements.

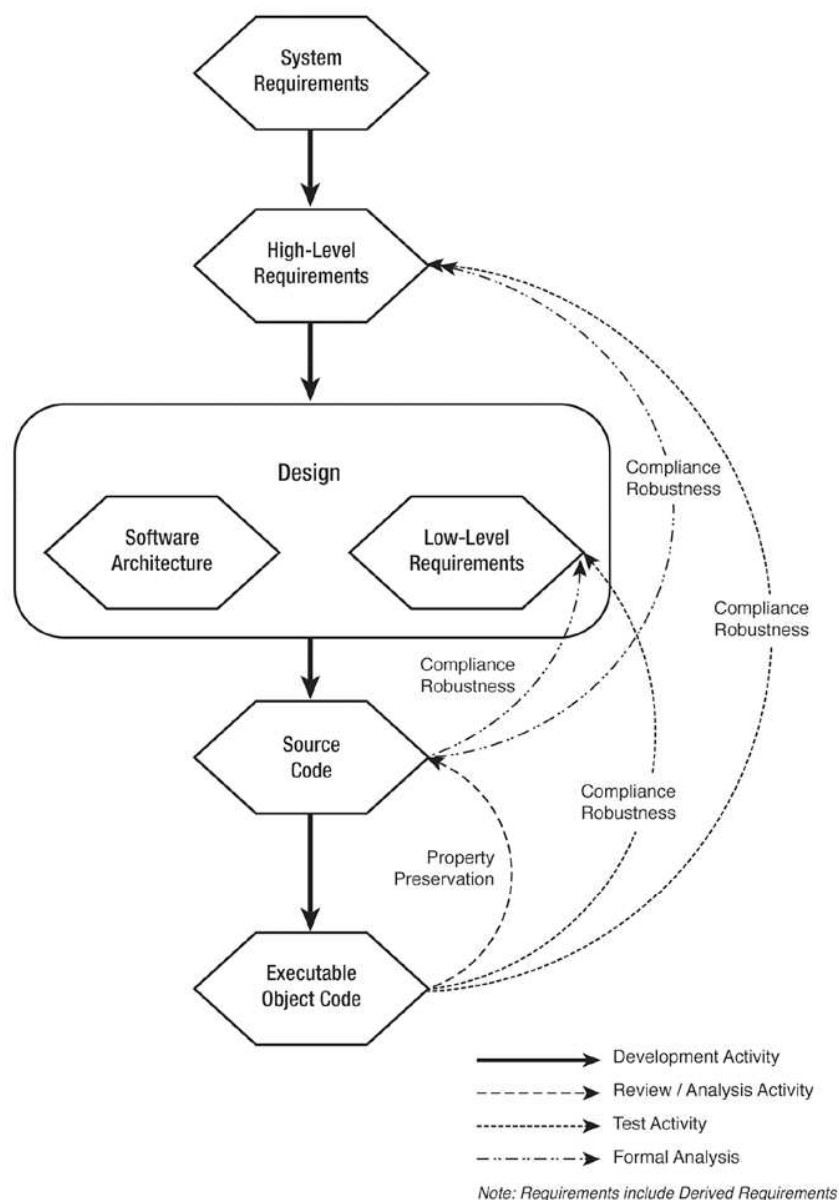


Figure FM.6-2 Possible Verification Paths for Executable Objects Code with Respect to Source Code and Requirements

FM.6.7.1 Principles of Coverage Analysis When Using Formal Methods

Test coverage analysis is a two step process, involving requirements-based coverage analysis and structural coverage analysis. Requirements-based coverage establishes that verification evidence exists for all of the requirements for the system. Structural coverage analysis, however, is driven out of the impracticality of achieving exhaustive testing and the consequent need to establish metrics to ensure that the testing performed is rigorous and sufficient. Unlike testing, the use of formal methods may provide an exhaustive assessment of the software. However, additional activities are required to achieve comparable coverage analysis.

When any low-level testing is used to verify low-level requirements for a software component, then the entire software component will be subject to DO-178C testing guidance for structural coverage analysis.

When only formal methods are used to verify requirements for a software component, sections FM.6.7.1.1-FM.6.7.1.5 describe activities that are required for coverage analysis of the component.

There are three objectives for coverage analysis. The first two objectives are for requirements-based coverage analysis which is very similar to the guidance in DO-178C and is detailed below in section FM.6.7.1.1:

- a. Coverage of high-level requirements is achieved.
- b. Coverage of low-level requirements is achieved.

The third objective is for structural coverage analysis:

- c. Verification coverage of software structure is achieved.

Structural coverage analysis is a notion strongly connected with testing and execution of code and is designed primarily to detect shortcomings in test. Verification by formal methods can be complete and exhaustive; hence equivalence to structural coverage is required rather than the analysis of how much structural coverage has been achieved. This equivalence is obtained by combining the activities described in sections FM.6.7.1.2 to FM.6.7.1.5 to detect:

- Shortcomings in requirements-based verification cases or procedures (covered in FM.6.7.1.2).
- Inadequacies in software requirements (covered in FM.6.7.1.3 and FM.6.7.1.4).
- Extraneous code, including dead code, and deactivated code (covered in FM.6.7.1.5).

FM.6.7.1.1 Requirements-Based Coverage Analysis

The purpose of requirements-based coverage analysis is to determine how well the implementation of the software requirements has been verified. This coverage analysis may reveal the need for additional requirements-based verification. Activities include:

- a. Analysis, using the associated Trace Data, to confirm that formal analysis cases exist for each software requirement.
- b. Analysis to confirm that formal analysis cases satisfy the criteria of normal range and robustness criteria as defined in sections FM.6.7, items a and b.
- c. Resolution of any deficiencies identified in the analysis. Possible solutions are adding or enhancing formal analysis cases.

FM.6.7.1.2 Complete Coverage of Each Requirement

The purpose is to ensure that all assumptions made during the formal analysis are verified. If the assumptions are all verified, then the formal analysis can give complete coverage of each requirement.

FM.6.7.1.3 Completeness of the Set of Requirements

Where requirements are formally modeled, it should be demonstrated that the set of requirements is complete with respect to the intended functions. That is:

- For all input conditions, the required output has been specified.
- For all outputs, the required input conditions have been specified.

If the set of requirements is found to be incomplete, then additional requirements (generally not derived requirements) should be added. If a demonstration of completeness cannot be achieved, then structural coverage analysis should be used.

FM.6.7.1.4 Detection of Unintended Dataflow Relationships

Verifying that the information flow in the Source Code complies with the requirements ensures that there are no unintended dependencies between the inputs and outputs of the code, and therefore ensures that each element in the Source Code is specified by requirements. If unintended dependencies exist then these should be resolved either by adding the missing requirements (generally not derived requirements) or removing the erroneous code.

FM.6.7.1.5 Detection of Extraneous Code, including Dead Code, and Deactivated Code

In addition to the formal method used, extraneous code, including dead code, and deactivated code should be identified by review or analysis (other than formal). Dead code should be removed and an analysis performed to assess the effect and the need for reverification. For deactivated code which is not intended to be executed in any configuration used within an aircraft or engine, a combination of analysis and testing should show that the means by which such code could be inadvertently executed are prevented, isolated, or eliminated. For deactivated code which is only executed in certain configurations of the target computer environment, the operational configuration needed for normal execution of this code should be established and additional verification cases and procedures developed to satisfy the required coverage objectives. Extraneous code should be removed or additional requirements specified to support the identified function or module.

FM.6.7.2 Reviews and Analyses of the Formal Analysis Cases, Procedures, and Results for the Executable Object Code

The review and analysis activities detect and report errors that may have been introduced within the formal analysis data. These reviews and analyses confirm that the formal analyses satisfy these objectives:

- a. Formal analysis cases: The objective is to provide evidence that the formal analysis achieves the coverage required for the applicable software level. All assumptions that were included in the formal analysis should be justified and any false assumptions (which would invalidate the analysis) should be identified and addressed. Formal

analysis cases should be reviewed against the criteria given in section FM.6.7, items a and b.

- b. Formal analysis procedures: The objective is to verify that the formal analysis cases were correctly developed into formal analysis procedures and expected results.
- c. Formal analysis results: The objective is to ensure that the formal analysis results are correct and that discrepancies between actual and expected results are explained.

This Page Intentionally Left Blank

FM.7.0 SOFTWARE CONFIGURATION MANAGEMENT PROCESS

Section 7 of DO-178C is unchanged.

This Page Intentionally Left Blank

FM.8.0 SOFTWARE QUALITY ASSURANCE PROCESS

Section 8 of DO-178C is unchanged.

This Page Intentionally Left Blank

FM.9.0 CERTIFICATION LIAISON PROCESS

Section 9 of DO-178C is unchanged.

This Page Intentionally Left Blank

FM.10.0 OVERVIEW OF CERTIFICATION PROCESS

Section 10 of DO-178C is unchanged.

This Page Intentionally Left Blank

FM.11.0 SOFTWARE LIFE CYCLE DATA

DO-178C sections 11.2, 11.4, 11.5, 11.9-11.12, and 11.14-22 are unchanged.

FM.11.1 Plan for Software Aspects of Certification

DO-178C section 11.1 is unchanged except for the following:

- d. *Software life cycle:* This section defines the software life cycle to be used and includes a summary of each of the software life cycle processes for which detailed information is defined in their respective software plans. The software life cycle processes should accommodate the technology from applicable supplements. The summary explains how the objectives of each software life cycle process (DO-178C objectives as well as applicable supplement objectives) will be satisfied, and specifies the organizations to be involved, the organizational responsibilities, and the system life cycle processes and certification liaison process responsibilities. If there are any conflicts between objectives either of DO-178C and/or supplements, an approach to resolve the conflict should be included.
- e. *Software life cycle data:* This section specifies the software life cycle data that will be produced and controlled by the software life cycle processes. This section also describes the relationship of the data to each other or to other data defining the system, including data produced in accordance with any supplements used, the software life cycle data to be submitted to the certification authority, the form of the data, and the means by which the data will be made available to the certification authority.

FM.11.3 Software Verification Plan

DO-178C section 11.3 is unchanged except for the following:

- c. *Verification methods:*
 - 4. Formal methods, including how to establish the formal analysis case selection process, the formal analysis procedures to be used, and the formal analysis data to be produced.
- d. *Verification environment:* A description of the equipment for testing, the testing and analysis tools, and how to apply these tools and hardware test equipment. DO-178C section 4.4.3, item b provides guidance on indicating target computer and simulator or emulator differences. For the formal part of the verification, a description of the formal methods and formal analysis tools used, and how to apply these tools.

FM.11.6 Software Requirements Standards

DO-178C section 11.6 is unchanged except for the following:

- b. *Notations to be used to express requirements, such as data flow diagrams and formal notations.* Notations used to define formal models should have mathematically defined syntax and semantics. All constraints on the notations needed to be compatible with formal methods should be included.

FM.11.7 Software Design Standards

DO-178C section 11.7 is unchanged except for the following:

- a. *Design description method(s) to be used.* Design description methods used to define formal models should have mathematically defined syntax and semantics. All constraints on the notations needed to be compatible with formal methods should be included.

FM.11.8 Software Code Standards

DO-178C section 11.8 is unchanged except for the following:

- a. *Programming language(s) to be used and/or defined subset(s).* For a programming language, reference the data that unambiguously defines the syntax, the control behavior, the data behavior, and side-effects of the language. This may require limiting the use of some features of a language. Where programming languages are used as the basis for formal analysis, they should have mathematically defined syntax and semantics. All programming language(s) constraints needed for the language(s) to be compatible with formal methods should be included.

FM.11.13 Software Verification Cases and Procedures

DO-178C section 11.13 is unchanged except for the following:

- d. Formal analysis cases: The purpose of each analysis case, the properties to be verified, and all assumptions used in the analysis.
- e. Formal analysis procedures: The step-by-step instructions for how each formal analysis case is to be setup and performed, how the analysis results are evaluated, and the environment of the formal analysis.

FM.12.0 ADDITIONAL CONSIDERATIONS

DO-178C section 12 is unchanged except for the addition of the following alternative method which can be applied to this supplement.

FM.12.3.5 Coverage Analysis When Using a Combination of Formal Methods and Testing

At the time this supplement was written, establishing coverage for a software component had only been demonstrated using either a formal method approach or a test based approach, but not a combination of the two. If approaches mature for deriving coverage when a combination of formal methods and testing are used to meet any objective for verification of Executable Object Code, different objectives and activities will be needed to ensure that coverage is achieved. In this case, the guidance below should be considered in developing objectives to replace those currently in Table FM.A-7 FM 3, 4, and 5-8. This approach will have to be agreed by the relevant certification authorities.

Even when a combination of formal methods and testing are used, functional tests executed in target hardware are always required to ensure that the software in the target computer will satisfy the high-level requirements as defined in DO-178C section 6.4.3, item c.

FM.12.3.5.1 Requirements-Based Coverage Analysis

The following guidance replaces the guidance in section FM.6.7.1.1 if a combination of formal methods and testing is used to verify the Executable Object Code.

The goal of requirements-based coverage analysis is to determine how well the implementation of the software requirements has been verified. This coverage analysis may reveal the need for additional requirements-based verification. The requirements-based coverage analysis should show that:

- a. Tests cases or formal analysis cases exist for each software requirement.
- b. The combination of tests cases and formal analysis cases satisfies the criteria of normal range and robustness criteria.

FM.12.3.5.2 Structural Coverage Analysis

The following guidance replaces sections FM.6.7.1.2 to FM.6.7.1.5 if a combination of formal methods and testing is used to verify the Executable Object Code.

The goal of this analysis is to determine which code structure was not covered by the combination of requirements-based test procedures and of requirements-based formal analysis procedures. The combination of requirements-based test cases and formal analysis cases may not have completely covered the code structure, so structural coverage analysis is performed and additional verification produced to provide structural coverage. Guidance includes:

- a. The analysis should confirm the degree of structural coverage appropriate to the software level.
- b. The structural coverage analysis can be performed on the Source Code, unless the software level is A and the compiler generates object code that is not directly traceable to Source Code statements. Then, additional verification should be

performed on the object code to establish the correctness of such generated code sequences. A compiler-generated array-bound check in the object code is an example of object code that is not directly traceable to the Source Code.

- c. The analysis should confirm the data coupling and control coupling between the code components.

The resolution of structural coverage shortfalls will either be by the test based approach (guidance given in DO-178C section 6.4.4.3) or the formal methods approach (guidance given in sections FM.6.7.1.2-5).

ANNEX FM.A – PROCESS OBJECTIVES AND OUTPUTS BY SOFTWARE LEVEL IN DO-178C

This annex provides tables that describe the objectives, related activities, and outputs given in DO-178C Annex A that are relevant when using formal methods in the production of airborne software.

The tables include guidance for:

- a. The process objectives applicable for each software level. For level E software, see DO-178C section 2.3.3.*
- b. The independence by software level of the software life cycle process activities applicable to satisfy that process's objectives.*
- c. The control category by software level for the software life cycle data produced by the software life cycle process activities (DO-178C section 7.3).*

The tables include references as appropriate to DO-178C and to this supplement. For clear identification, references to DO-178C do not include a prefix, whereas references to this supplement include “FM.” as a prefix.

These tables should not be used as a checklist. In order to fully understand the guidance, the full body of DO-178C and this supplement should be considered.

The following legend applies to “Applicability by Software Level” and “Control Category by Software Level” for all tables:

LEGEND:	● <i>The objective should be satisfied with independence.</i> ○ <i>The objective should be satisfied.</i>
<i>Blank</i>	<i>Satisfaction of objective is at applicant's discretion.</i> <u>Note:</u> Where an objective is in gray text and is blank against all software levels, this means that meeting the guidance given in this supplement mitigates the objective entirely.
①	<i>Data satisfies the objectives of Control Category 1 (CC1).</i>
②	<i>Data satisfies the objectives of Control Category 2 (CC2).</i>

Table FM.A-1 Software Planning Process

	Objective		Activity	Applicability by Software Level				Output		Control Category by Software Level			
	Description	Ref	Ref	A	B	C	D	Data Item	Ref	A	B	C	D
1	The activities of the software life cycle processes are defined.	4.1.a	4.2.a 4.2.c 4.2.d 4.2.e 4.2.g 4.2.i 4.2.l 4.3.c FM.4.3	○	○	○	○	PSAC SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	① ② ② ② ②
2	The software life cycle(s), including the inter-relationships between the processes, their sequencing, feedback mechanisms, and transition criteria, is defined.	4.1.b	4.2.i 4.3.b FM.4.3	○	○	○		PSAC SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	
3	Software life cycle environment is selected and defined.	4.1.c	4.4.1 4.4.2.a 4.4.2.b 4.4.2.c 4.4.3	○	○	○		PSAC SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	
4	Additional considerations are addressed.	4.1.d	4.2.f 4.2.h 4.2.i 4.2.j 4.2.k	○	○	○	○	PSAC SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	① ② ② ② ②
5	Software development standards are defined.	4.1.e	4.2.b 4.2.g 4.5	○	○	○		SW Requirements Standards SW Design Standards SW Code Standards	FM.11.6 FM.11.7 FM.11.8	① ① ①	① ① ①	② ② ②	
6	Software plans comply with this document.	4.1.f	4.3.a 4.6	○	○	○		Software Verification Results	11.14	②	②	②	
7	Development and revision of software plans are coordinated.	4.1.g	4.2.g 4.6	○	○	○		Software Verification Results	11.14	②	②	②	

Table FM.A-2 Software Development Processes

DO-178C Table A-2 is unchanged.

Table FM.A-3 Verification of Outputs of Software Requirements Process

	Objective		Activity	Applicability by Software Level				Output		Control Category by Software Level			
	Description	Ref		Ref	A	B	C	D	Data Item	Ref	A	B	C
1	High-level requirements comply with system requirements.	FM.6.3.a FM.6.3.1.a	FM.6.3.1	●	●	○	○	Software Verification Results	11.14	②	②	②	②
2	High-level requirements are accurate and consistent.	FM.6.3.b FM.6.3.c FM.6.3.1.b	FM.6.3.1	●	●	○	○	Software Verification Results	11.14	②	②	②	②
3	High-level requirements are compatible with target computer.	FM.6.3.d FM.6.3.1.c	FM.6.3.1	○	○			Software Verification Results	11.14	②	②		
4	High-level requirements are verifiable.	FM.6.3.e FM.6.3.1.d	FM.6.3.1	○	○	○		Software Verification Results	11.14	②	②	②	
5	High-level requirements conform to standards.	FM.6.3.f FM.6.3.1.e	FM.6.3.1	○	○	○		Software Verification Results	11.14	②	②	②	
6	High-level requirements are traceable to system requirements.	FM.6.3.g FM.6.3.1.f	FM.6.3.1	○	○	○	○	Software Verification Results	11.14	②	②	②	②
7	Algorithms are accurate.	FM.6.3.h FM.6.3.1.g	FM.6.3.1	●	●	○		Software Verification Results	11.14	②	②	②	
FM 8	Formal analysis cases and procedures are correct.	FM.6.3.6.a FM.6.3.6.b	FM.6.3.6	●	○	○		Software Verification Results	11.14	②	②	②	
FM 9	Formal analysis results are correct and discrepancies explained.	FM.6.3.6.c	FM.6.3.6	●	○	○		Software Verification Results	11.14	②	②	②	
FM 10	Requirements formalization is correct.	FM.6.3.i	FM.6.3.i	●	●	○		Software Verification Results	11.14	②	②	②	
FM 11	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	Software Verification Results	11.14	②	②	②	②

Table FM.A-4 Verification of Outputs of Software Design Process

	Objective		Activity	Applicability by Software Level				Output		Control Category by Software Level			
	Description	Ref		A	B	C	D	Data Item	Ref	A	B	C	D
1	Low-level requirements comply with high-level requirements.	FM.6.3.a FM.6.3.2.a	FM.6.3.2	●	●	○		Software Verification Results	11.14	②	②	②	
2	Low-level requirements are accurate and consistent.	FM.6.3.b FM.6.3.c FM.6.3.2.b	FM.6.3.2	●	●	○		Software Verification Results	11.14	②	②	②	
3	Low-level requirements are compatible with target computer.	FM.6.3.d FM.6.3.2.c	FM.6.3.2	○	○			Software Verification Results	11.14	②	②		
4	Low-level requirements are verifiable.	FM.6.3.e FM.6.3.2.d	FM.6.3.2	○	○			Software Verification Results	11.14	②	②		
5	Low-level requirements conform to standards.	FM.6.3.f FM.6.3.2.e	FM.6.3.2	○	○	○		Software Verification Results	11.14	②	②	②	
6	Low-level requirements are traceable to high-level requirements.	FM.6.3.g FM.6.3.2.f	FM.6.3.2	○	○	○		Software Verification Results	11.14	②	②	②	
7	Algorithms are accurate.	FM.6.3.h FM.6.3.2.g	FM.6.3.2	●	●	○		Software Verification Results	11.14	②	②	②	
8	Software architecture is compatible with high-level requirements.	FM.6.3.3.a	FM.6.3.3	●	○	○		Software Verification Results	11.14	②	②	②	
9	Software architecture is consistent.	FM.6.3.c FM.6.3.3.b	FM.6.3.3	●	○	○		Software Verification Results	11.14	②	②	②	
10	Software architecture is compatible with target computer.	FM.6.3.d FM.6.3.3.c	FM.6.3.3	○	○			Software Verification Results	11.14	②	②		
11	Software architecture is verifiable.	FM.6.3.e FM.6.3.3.d	FM.6.3.3	○	○			Software Verification Results	11.14	②	②		
12	Software architecture conforms to standards.	FM.6.3.f FM.6.3.3.e	FM.6.3.3	○	○	○		Software Verification Results	11.14	②	②	②	
13	Software partitioning integrity is confirmed.	FM.6.3.3.f	FM.6.3.3	●	○	○	○	Software Verification Results	11.14	②	②	②	②
FM 14	Formal analysis cases and procedures are correct.	FM.6.3.6.a FM.6.3.6.b	FM.6.3.6	●	○	○		Software Verification Results	11.14	②	②	②	
FM 15	Formal analysis results are correct and discrepancies explained.	FM.6.3.6.c	FM.6.3.6	●	○	○		Software Verification Results	11.14	②	②	②	
FM 16	Requirements formalization is correct.	FM.6.3.i	FM.6.3.i	●	●	○		Software Verification Results	11.14	②	②	②	
FM 17	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	Software Verification Results	11.14	②	②	②	②

Table FM.A-5 Verification of Outputs of Software Coding & Integration Processes

	Objective		Activity Ref	Applicability by Software Level				Output		Control Category by Software Level			
	Description	Ref		A	B	C	D	Data Item	Ref	A	B	C	D
1	Source Code complies with low-level requirements.	FM.6.3.a FM.6.3.4.a	FM.6.3.4	●	●	○		Software Verification Results	11.14	②	②	②	
2	Source Code complies with software architecture.	FM.6.3.a FM.6.3.4.b	FM.6.3.4	●	○	○		Software Verification Results	11.14	②	②	②	
3	Source Code is verifiable.	FM.6.3.e FM.6.3.4.c	FM.6.3.4	○	○			Software Verification Results	11.14	②	②		
4	Source Code conforms to standards.	FM.6.3.f FM.6.3.4.d	FM.6.3.4	○	○	○		Software Verification Results	11.14	②	②	②	
5	Source Code is traceable to low-level requirements.	FM.6.3 FM.6.3.4.e	FM.6.3.4	○	○	○		Software Verification Results	11.14	②	②	②	
6	Source Code is accurate and consistent.	FM.6.3.b FM.6.3.c FM.6.3.4.f	FM.6.3.4	●	○	○		Software Verification Results	11.14	②	②	②	
7	Output of software integration process is complete and correct.	6.3.5.a	6.3.5	○	○	○		Software Verification Results	11.14	②	②	②	
8	Parameter Data Item File is correct and complete.	6.6.a	6.6	●	●	○	○	Software Verification Cases and Procedures	11.13	①	①	②	②
								Software Verification Results	11.14	②	②	②	②
9	Verification of Parameter Data Item File is achieved.	6.6.b	6.6	●	●	○		Software Verification Results	11.14	②	②	②	
FM 10	Formal analysis cases and procedures are correct.	FM.6.3.6.a FM.6.3.6.b	FM.6.3.6	●	○	○		Software Verification Results	11.14	②	②	②	
FM 11	Formal analysis results are correct and discrepancies explained.	FM.6.3.6.c	FM.6.3.6	●	○	○		Software Verification Results	11.14	②	②	②	
FM 12	Requirements formalization is correct.	FM.6.3.i	FM.6.3.i	●	●	○		Software Verification Results	11.14	②	②	②	
FM 13	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	Software Verification Results	11.14	②	②	②	②

Table FM.A-6 Testing of Outputs of Integration Process

	Objective		Activity	Applicability by Software Level				Output		Control Category by Software Level			
	Description	Ref	Ref	A	B	C	D	Data Item	Ref	A	B	C	D
1	Executable Object Code complies with high-level requirements.	FM.6.7.a FM.6.7.c	FM.6.7 FM.6.5	○	○	○	○	Software Verification Cases and Procedures	FM.11.13	①	①	②	②
								Software Verification Results	11.14	②	②	②	②
								Trace Data	11.21	①	①	②	②
2	Executable Object Code is robust with high-level requirements.	FM.6.7.b FM.6.7.c	FM.6.7 FM.6.5	○	○	○	○	Software Verification Cases and Procedures	FM.11.13	①	①	②	②
								Software Verification Results	11.14	②	②	②	②
								Trace Data	11.21	①	①	②	②
3	Executable Object Code complies with low-level requirements.	FM.6.7.d FM.6.7.c	FM.6.7 FM.6.5	●	●	○		Software Verification Cases and Procedures	FM.11.13	①	①	②	
								Software Verification Results	11.14	②	②	②	
								Trace Data	11.21	①	①	②	
4	Executable Object Code is robust with low-level requirements.	FM.6.7.b FM.6.7.c	FM.6.7 FM.6.5	●	○	○		Software Verification Cases and Procedures	FM.11.13	①	①	②	
								Software Verification Results	11.14	②	②	②	
								Trace Data	11.21	①	①	②	
5	Executable Object Code is compatible with target computer.	6.4.e FM.6.7.e	6.4.1.a 6.4.3.a FM.6.7	○	○	○	○	Software Verification Cases and Procedures	FM.11.13	①	①	②	②
								Software Verification Results	11.14	②	②	②	②

Table FM.A-7 Verification of Verification Process Results

	Objective		Activity	Applicability by Software Level				Output		Control Category by Software Level			
	Description	Ref		A	B	C	D	Data Item	Ref	A	B	C	D
1	Test procedures are correct.	6.4.5.b	6.4.5					Software Verification Results	11.14				
2	Test results are correct and discrepancies explained.	6.4.5.c	6.4.5					Software Verification Results	11.14				
3	Test coverage of high-level requirements is achieved.	6.4.4.a	6.4.4.1					Software Verification Results	11.14				
4	Test coverage of low-level requirements is achieved.	6.4.4.b	6.4.4.1					Software Verification Results	11.14				
5	Test coverage of software structure (modified condition/decision coverage) is achieved.	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3					Software Verification Results	11.14				
6	Test coverage of software structure (decision coverage) is achieved.	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3					Software Verification Results	11.14				
7	Test coverage of software structure (statement coverage) is achieved.	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3					Software Verification Results	11.14				
8	Test coverage of software structure (data coupling and control coupling) is achieved.	6.4.4.d	6.4.4.2.c 6.4.4.2.d 6.4.4.3					Software Verification Results	11.14				
9	Verification of additional code, that cannot be traced to Source Code, is achieved.	6.4.4.c	6.4.4.2.b					Software Verification Results	11.14				
FM 1	Formal analysis cases and procedures are correct.	FM.6.7.2.a FM.6.7.2.b	FM.6.7.2	●	○	○		Software Verification Results	11.14	②	②	②	
FM 2	Formal analysis results are correct and discrepancies explained.	FM.6.7.2.c	FM.6.7.2	●	○	○		Software Verification Results	11.14	②	②	②	
FM 3	Coverage of high-level requirements is achieved.	FM.6.7.1.a	FM.6.7.1.1	●	○	○	○	Software Verification Results	11.14	②	②	②	②
FM 4	Coverage of low-level requirements is achieved.	FM.6.7.1.b	FM.6.7.1.1	●	○	○		Software Verification Results	11.14	②	②	②	
FM 5-8 (see Note)	Verification coverage of software structure is achieved.	FM.6.7.1.c	FM.6.7.1.2 FM.6.7.1.3 FM.6.7.1.4 FM.6.7.1.5	●	●	○		Software Verification Results	11.14	②	②	②	
FM 9	Verification of property preservation between source and object code.	FM.6.7.f	FM.6.7	●	●	○	○	Software Verification Results	11.14	②	②	②	②
FM 10	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	Software Verification Results	11.14	②	②	②	②

Note: FM 5-8 is a single objective, replacing objectives 5 through 8 in DO-178C Table A-7.

Table FM.A-8 Software Configuration Management Process

DO-178C Table A-8 is unchanged.

Table FM.A-9 Software Quality Assurance Process

DO-178C Table A-9 is unchanged.

Table FM.A-10 Certification Liaison Process

DO-178C Table A-10 is unchanged.

This Page Intentionally Left Blank

The acronyms and glossary definitions provided in this annex are for the terms used in this document. The definitions defined in Annex B of DO-178C are not repeated here.

Acronym	Meaning
CTL	Computation Tree Logic
DU	Display Unit
HLR	High-Level Requirements
LLR	Low-Level Requirements
UAV	Unmanned Air Vehicle
WM	Window Manager

GLOSSARY

Conservative representation – A simplified or abstract representation in a formal notation of life cycle data (such as requirements or code) such that statements that are true for this representation will always be true for the life cycle data itself. In many cases, analysis of a conservative representation will yield pessimistic results but never unsound ones. The representation chosen simplifies or makes possible the analysis.

Formal analysis – The use of mathematical reasoning to guarantee that properties are always satisfied by a formal model.

Formal analysis case – A combination of the property to be analyzed or proved, all assumptions for the formal analysis, and expected results, such as to verify correct functionality and to establish conditions that reveal potential errors; analogous to a test case.

Formal methods – Descriptive notations and analytical methods used to construct, develop and reason about mathematical models of system behavior. A formal method is a formal analysis carried out on a formal model.

Formal model (and Model) – A model is an abstract representation of a given set of aspects of a system that is used for analysis, simulation, code generation, or any combination thereof. A formal model is a model defined using a formal notation.

Formal notation – A notation having a precise, unambiguous, mathematically defined syntax and semantics.

Information flow – A description of the dependencies between an output of a software module and its inputs. Where information flow is stated as part of a set of requirements, the code can be inspected to ensure it fully conforms to these requirements (that is, all outputs are defined, each output does in fact use all of its defined input parameters and that there are no additional input parameters on which the output depends).

Property – The representation of a requirement (functional or non-functional) in a formal notation.

Property preservation – Applies to properties when software artifacts are translated from a source language to a target language. Property preservation is ensured when any property that holds on a source artifact also holds on its translated counterpart.

Semantics – The meaning of expressions or how they are evaluated when executed.

Soundness – A characteristic of a logical argument or method that holds if and only if it is impossible to reach a false conclusion from true statements. A sound method is one that never permits a property to be declared true when it is not true.

Syntax – Structure of expressions or the rules for putting symbols together to form an expression.

Well-formed expression – An expression that only contains elements from a defined set of primitive operands and operators and meets all of the rules for valid combinations of these primitives.

PROCESS OBJECTIVES AND OUTPUTS BY ASSURANCE LEVEL IN DO-278A

This annex provides tables that describe the objectives, related activities, and outputs given in DO-278A Annex A relevant when using formal methods in the production of software for Communication, Navigation, Surveillance, and Air Traffic Management (CNS/ATM) systems. The tables include guidance for:

- a. The process objectives applicable for assurance levels (AL) 1-5, as defined in DO-278A section 2.3.2.
- b. The independence by assurance level of the software life cycle process activities applicable to satisfy that process's objectives.
- c. The control category by assurance level for the software life cycle data produced by the software life cycle process activities (DO-278A section 7.3).

The tables include references as appropriate to DO-278A and to this document. For clear identification, references to DO-278A do not include a prefix, whereas references to this document include "FM." as a prefix.

These tables should not be used as a checklist. In order to fully understand the guidance, the full body of DO-278A and this document should be considered. When reviewing this document for CNS/ATM systems, the following terms specific to the airborne community should be replaced as follows:

- "airborne" with "CNS/ATM system"
- "certification" with "approval"
- "airworthiness requirements" with "applicable approval requirements"
- "certification liaison process" with "approval process"
- "Plan for Software Aspects of Certification" (PSAC) with "Plan for Software Aspects of Approval" (PSAA)

This document does not provide guidance for the use of formal methods for commercial off-the-shelf (COTS) software or adaptation parameter data (DO-278A sections 12.4 and 12.5).

The following legend applies to "Applicability by Assurance Level" and "Control Category by Assurance Level" for all tables:

LEGEND:	● The objective should be satisfied with independence.
	○ The objective should be satisfied.
Blank	Satisfaction of objective is at applicant's discretion.
	<u>Note:</u> Where an objective is in gray text and blank against all assurance levels, this means that meeting the guidance given in this document mitigates the objective entirely.
①	Data satisfies the objectives of Control Category 1 (CC1).
②	Data satisfies the objectives of Control Category 2 (CC2).

Table FM.C-1 Software Planning Process

	Objective		Activity	Applicability by Assurance Level					Output		Control Category by Assurance Level				
	Description	Ref	Ref	AL 1	AL 2	AL 3	AL 4	AL 5	Data Item	Ref	AL 1	AL 2	AL 3	AL 4	AL 5
1	The activities of the software life cycle processes are defined.	4.1.a	4.2.a 4.2.c 4.2.d 4.2.e 4.2.g 4.2.i 4.2.l 4.3.c FM.4.3	○	○	○	○	○	PSAA SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	① ② ② ② ②	① ② ② ② ②
2	The software life cycle(s), including the inter-relationships between the processes, their sequencing, feedback mechanisms, and transition criteria, is defined.	4.1.b	4.2.i 4.3.b FM.4.3	○	○	○	○		PSAA SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	① ② ② ② ②	
3	Software life cycle environment is selected and defined.	4.1.c	4.4.1 4.4.2.a 4.4.2.b 4.4.2.c 4.4.3	○	○	○	○		PSAA SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	① ② ② ② ②	
4	Additional considerations are addressed.	4.1.d	4.2.f 4.2.h 4.2.i 4.2.j 4.2.k	○	○	○	○	○	PSAA SDP SVP SCM Plan SQA Plan	FM.11.1 11.2 FM.11.3 11.4 11.5	① ① ① ① ①	① ① ① ① ①	① ② ② ② ②	① ② ② ② ②	① ② ② ② ②
5	Software development standards are defined.	4.1.e	4.2.b 4.2.g 4.5	○	○	○	○		SW Requirements Standards SW Design Standards SW Code Standards	FM.11.6 FM.11.7 FM.11.8	① ① ①	① ① ①	② ② ②	② ② ②	
6	Software plans comply with this document.	4.1.f	4.3.a 4.6	○	○	○	○		Software Verification Results	11.14	②	②	②	②	
7	Development and revision of software plans are coordinated.	4.1.g	4.2.g 4.6	○	○	○	○		Software Verification Results	11.14	②	②	②	②	

Table FM.C-2 Software Development Processes

DO-278A Table A-2 is unchanged.

Table FM.C-3 Verification of Outputs of Software Requirements Process

	Objective		Activity	Applicability by Assurance Level					Output		Control Category by Assurance Level				
	Description	Ref		Ref	AL1	AL2	AL3	AL4	AL5	Data Item	Ref	AL1	AL2	AL3	AL4
1	High-level requirements comply with system requirements.	FM.6.3.a FM.6.3.1.a	FM.6.3.1	●	●	○	○	○	Software Verification Results	11.14	②	②	②	②	②
2	High-level requirements are accurate and consistent.	FM.6.3.b FM.6.3.c FM.6.3.1.b	FM.6.3.1	●	●	○	○	○	Software Verification Results	11.14	②	②	②	②	②
3	High-level requirements are compatible with target computer.	FM.6.3.d FM.6.3.1.c	FM.6.3.1	○	○				Software Verification Results	11.14	②	②			
4	High-level requirements are verifiable.	FM.6.3.e FM.6.3.1.d	FM.6.3.1	○	○	○	○		Software Verification Results	11.14	②	②	②	②	
5	High-level requirements conform to standards.	FM.6.3.f FM.6.3.1.e	FM.6.3.1	○	○	○	○		Software Verification Results	11.14	②	②	②	②	
6	High-level requirements are traceable to system requirements.	FM.6.3.g FM.6.3.1.f	FM.6.3.1	○	○	○	○	○	Software Verification Results	11.14	②	②	②	②	②
7	Algorithms are accurate.	FM.6.3.h FM.6.3.1.g	FM.6.3.1	●	●	○	○		Software Verification Results	11.14	②	②	②	②	
FM 8	Formal analysis cases and procedures are correct.	FM.6.3.6.a FM.6.3.6.b	FM.6.3.6	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 9	Formal analysis results are correct and discrepancies explained.	FM.6.3.6.c	FM.6.3.6	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 10	Requirements formalization is correct.	FM.6.3.i	FM.6.3.i	●	●	○	○		Software Verification Results	11.14	②	②	②	②	
FM 11	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	○	Software Verification Results	11.14	②	②	②	②	②

Table FM.C-4 Verification of Outputs of Software Design Process

	Objective		Activity	Applicability by Assurance Level					Output		Control Category by Assurance Level				
	Description	Ref		AL 1	AL 2	AL 3	AL 4	AL 5	Data Item	Ref	AL 1	AL 2	AL 3	AL 4	AL 5
1	Low-level requirements comply with high-level requirements.	FM.6.3.a FM.6.3.2.a	FM.6.3.2	●	●	○			Software Verification Results	11.14	②	②	②		
2	Low-level requirements are accurate and consistent.	FM.6.3.b FM.6.3.c FM.6.3.2.b	FM.6.3.2	●	●	○			Software Verification Results	11.14	②	②	②		
3	Low-level requirements are compatible with target computer.	FM.6.3.d FM.6.3.2.c	FM.6.3.2	○	○				Software Verification Results	11.14	②	②			
4	Low-level requirements are verifiable.	FM.6.3.e FM.6.3.2.d	FM.6.3.2	○	○				Software Verification Results	11.14	②	②			
5	Low-level requirements conform to standards.	FM.6.3.f FM.6.3.2.e	FM.6.3.2	○	○	○			Software Verification Results	11.14	②	②	②		
6	Low-level requirements are traceable to high-level requirements.	FM.6.3.g FM.6.3.2.f	FM.6.3.2	○	○	○			Software Verification Results	11.14	②	②	②		
7	Algorithms are accurate.	FM.6.3.h FM.6.3.2.g	FM.6.3.2	●	●	○	○		Software Verification Results	11.14	②	②	②	②	
8	Software architecture is compatible with high-level requirements.	FM.6.3.3.a	FM.6.3.3	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
9	Software architecture is consistent.	FM.6.3.c FM.6.3.3.b	FM.6.3.3	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
10	Software architecture is compatible with target computer.	FM.6.3.d FM.6.3.3.c	FM.6.3.3	○	○				Software Verification Results	11.14	②	②			
11	Software architecture is verifiable.	FM.6.3.e FM.6.3.3.d	FM.6.3.3	○	○				Software Verification Results	11.14	②	②			
12	Software architecture conforms to standards.	FM.6.3.f FM.6.3.3.e	FM.6.3.3	○	○	○	○		Software Verification Results	11.14	②	②	②	②	
13	Software partitioning integrity is confirmed.	FM.6.3.3.f	FM.6.3.3	●	○	○	○	○	Software Verification Results	11.14	②	②	②	②	②
FM 14	Formal analysis cases and procedures are correct.	FM.6.3.6.a FM.6.3.6.b	FM.6.3.6	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 15	Formal analysis results are correct and discrepancies explained.	FM.6.3.6.c	FM.6.3.6	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 16	Requirements formalization is correct.	FM.6.3.i	FM.6.3.i	●	●	○	○		Software Verification Results	11.14	②	②	②	②	
FM 17	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	○	Software Verification Results	11.14	②	②	②	②	②

Table FM.C-5 Verification of Outputs of Software Coding & Integration Processes

	Objective		Activity	Applicability by Assurance Level					Output		Control Category by Assurance Level				
	Description	Ref	Ref	AL 1	AL 2	AL 3	AL 4	AL 5	Data Item	Ref	AL 1	AL 2	AL 3	AL 4	AL 5
1	Source Code complies with low-level requirements.	FM.6.3.a FM.6.3.4.a	FM.6.3.4	●	●	○			Software Verification Results	11.14	②	②	②		
2	Source Code complies with software architecture.	FM.6.3.a FM.6.3.4.b	FM.6.3.4	●	○	○			Software Verification Results	11.14	②	②	②		
3	Source Code is verifiable.	FM.6.3.e FM.6.3.4.c	FM.6.3.4	○	○				Software Verification Results	11.14	②	②			
4	Source Code conforms to standards.	FM.6.3.f FM.6.3.4.d	FM.6.3.4	○	○	○			Software Verification Results	11.14	②	②	②		
5	Source Code is traceable to low-level requirements.	FM.6.3 FM.6.3.4.e	FM.6.3.4	○	○	○			Software Verification Results	11.14	②	②	②		
6	Source Code is accurate and consistent.	FM.6.3.b FM.6.3.c FM.6.3.4.f	FM.6.3.4	●	○	○			Software Verification Results	11.14	②	②	②		
7	Output of software integration process is complete and correct.	6.3.5.a	6.3.5	○	○	○	○		Software Verification Results	11.14	②	②	②	②	
8	Adaptation Data Item File is correct and complete.	6.6.a	6.6	●	●	○	○	○	Software Verification Cases and Procedures	11.13	①	①	②	②	②
									Software Verification Results	11.14	②	②	②	②	②
9	Verification of Adaptation Data Item File is achieved.	6.6.b	6.6	●	●	○	○		Software Verification Results	11.14	②	②	②	②	
FM 10	Formal analysis cases and procedures are correct.	FM.6.3.6.a FM.6.3.6.b	FM.6.3.6	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 11	Formal analysis results are correct and discrepancies explained.	FM.6.3.6.c	FM.6.3.6	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 12	Requirements formalization is correct.	FM.6.3.i	FM.6.3.i	●	●	○	○		Software Verification Results	11.14	②	②	②	②	
FM 13	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	○	Software Verification Results	11.14	②	②	②	②	②

Table FM.C-6 Testing of Outputs of Integration Process

	Objective		Activity	Applicability by Assurance Level					Output		Control Category by Assurance Level				
	Description	Ref	Ref	AL 1	AL 2	AL 3	AL 4	AL 5	Data Item	Ref	AL 1	AL 2	AL 3	AL 4	AL 5
1	Executable Object Code complies with high-level requirements.	FM.6.7.a FM.6.7.c	FM.6.7 FM.6.5	○	○	○	○	○	Software Verification Cases and Procedures	FM.11.13	①	①	②	②	②
									Software Verification Results	11.14	②	②	②	②	②
									Trace Data	11.21	①	①	②	②	②
2	Executable Object Code is robust with high-level requirements.	FM.6.7.b FM.6.7.c	FM.6.7 FM.6.5	○	○	○	○	○	Software Verification Cases and Procedures	FM.11.13	①	①	②	②	②
									Software Verification Results	11.14	②	②	②	②	②
									Trace Data	11.21	①	①	②	②	②
3	Executable Object Code complies with low-level requirements.	FM.6.7.d FM.6.7.c	FM.6.7 FM.6.5	●	●	○			Software Verification Cases and Procedures	FM.11.13	①	①	②		
									Software Verification Results	11.14	②	②	②		
									Trace Data	11.21	①	①	②		
4	Executable Object Code is robust with low-level requirements.	FM.6.7.b FM.6.7.c	FM.6.7 FM.6.5	●	○	○			Software Verification Cases and Procedures	FM.11.13	①	①	②		
									Software Verification Results	11.14	②	②	②		
									Trace Data	11.21	①	①	②		
5	Executable Object Code is compatible with target computer.	6.4.e FM.6.7.e	6.4.1.a 6.4.3.a FM.6.7	○	○	○	○	○	Software Verification Cases and Procedures	FM.11.13	①	①	②	②	②
									Software Verification Results	11.14	②	②	②	②	②

Table FM.C-7 Verification of Verification Process Results

	Objective		Activity	Applicability by Assurance Level					Output		Control Category by Assurance Level				
	Description	Ref	Ref	AL1	AL2	AL3	AL4	AL5	Data Item	Ref	AL1	AL2	AL3	AL4	AL5
1	Test procedures are correct.	6.4.5.b	6.4.5						Software Verification Results	11.14					
2	Test results are correct and discrepancies explained.	6.4.5.c	6.4.5						Software Verification Results	11.14					
3	Test coverage of high-level requirements is achieved.	6.4.4.a	6.4.4.1						Software Verification Results	11.14					
4	Test coverage of low-level requirements is achieved.	6.4.4.b	6.4.4.1						Software Verification Results	11.14					
5	Test coverage of software structure (modified condition/decision) is achieved.	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3						Software Verification Results	11.14					
6	Test coverage of software structure (decision coverage) is achieved.	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3						Software Verification Results	11.14					
7	Test coverage of software structure (statement coverage) is achieved.	6.4.4.c	6.4.4.2.a 6.4.4.2.b 6.4.4.2.d 6.4.4.3						Software Verification Results	11.14					
8	Test coverage of software structure (data coupling and control coupling) is achieved.	6.4.4.d	6.4.4.2.c 6.4.4.2.d 6.4.4.3						Software Verification Results	11.14					
9	Verification of additional code, that cannot be traced to Source Code, is achieved.	6.4.4.c	6.4.4.2.b						Software Verification Results	11.14					
FM 1	Formal analysis cases and procedures are correct.	FM.6.7.2.a FM.6.7.2.b	FM.6.7.2	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 2	Formal analysis results are correct and discrepancies explained.	FM.6.7.2.c	FM.6.7.2	●	○	○	○		Software Verification Results	11.14	②	②	②	②	
FM 3	Coverage of high-level requirements is achieved.	FM.6.7.1.a	FM.6.7.1.1	●	○	○	○	○	Software Verification Results	11.14	②	②	②	②	②
FM 4	Coverage of low-level requirements is achieved.	FM.6.7.1.b	FM.6.7.1.1	●	○	○			Software Verification Results	11.14	②	②	②		
FM 5-8 (see Note)	Verification coverage of software structure is achieved.	FM.6.7.1.c	FM.6.7.1.2 FM.6.7.1.3 FM.6.7.1.4 FM.6.7.1.5	●	●	○	○		Software Verification Results	11.14	②	②	②	②	
FM 9	Verification of property preservation between source and object code.	FM.6.7.f	FM.6.7	●	●	○	○	○	Software Verification Results	11.14	②	②	②	②	②
FM 10	Formal method is correctly defined, justified, and appropriate.	FM.6.2.1	FM.6.2.1.a FM.6.2.1.b FM.6.2.1.c	●	○	○	○	○	Software Verification Results	11.14	②	②	②	②	②

Note: FM 5-8 is a single objective, replacing objectives 5 through 8 in DO-278A Table A-7.

Table FM.C-8 Software Configuration Management Process

DO-278A Table A-8 is unchanged.

Table FM.C-9 Software Quality Assurance Process

DO-278A Table A-9 is unchanged.

Table FM.C-10 Software Approval Process

DO-278A Table A-10 is unchanged.

This Page Intentionally Left Blank

APPENDIX FM.A COMMITTEE MEMBERSHIP

EXECUTIVE COMMITTEE MEMBERS

Jim Krodel, Pratt & Whitney	SC-205 Chair
Gérard Ladier, Airbus/Aerospace Valley	WG-71 Chair
Mike DeWalt, Certification Services, Inc./FAA	SC-205 Secretary (until March 2008)
Leslie Alford, Boeing Company	SC-205 Secretary (from March 2008)
Ross Hannan, Sigma Associates (Aerospace)	WG-71 Secretary
Barbara Lingberg, FAA	FAA Representative/CAST Chair
Jean-Luc Delamaide, EASA	EASA Representative
John Coleman, Dawson Consulting	Sub-group Liaison
Matt Jaffe, Embry-Riddle Aeronautical University	Web Site Liaison
Todd R. White, L-3 Communications/Qualtech	Collaborative Technology Software Liaison

SUB-GROUP LEADERSHIP

SG-1 – Document Integration

Ron Ashpole, SILVER ATENA	SG-1 Co-chair
Tom Ferrell, Ferrell and Associates Consulting	SG-1 Co-chair (until March 2008)
Marty Gasiorowski, Worldwide Certification Services	SG-1 Co-chair (from March 2008)
Tom Roth, Airborne Software Certification Consulting	SG-1 Secretary

SG-2 – Issues and Rationale

Ross Hannan, Sigma Associates (Aerospace)	SG-2 Co-chair
Mike DeWalt, Certification Services, Inc./FAA	SG-2 Co-chair (until March 2008)
Will Struck, FAA	SG-2 Co-chair (until March 2009)
Fred Moyer, Rockwell Collins	SG-2 Co-chair (from April 2009)
John Angermayer, Mitre	SG-2 Secretary

SG-3 – Tool Qualification

Frédéric Pothon, ACG Solutions	SG-3 Co-chair
Leanna Rierson, Digital Safety Consulting	SG-3 Co-chair
Bernard Dion, Esterel Technologies	SG-3 Co-secretary
Gene Kelly, CertTech	SG-3 Co-secretary (until May 2009)
Mo Piper, Boeing Company	SG-3 Co-secretary (from May 2009)

SG-4 – Model-Based Development and Verification

Pierre Lionne, EADS APSYS	SG-4 Co-chair
Mark Lillis, Goodrich GPECS	SG-4 Co-chair
Hervé Delseny, Airbus	SG-4 Co-chair
Martha Blankenberger, Rolls-Royce	SG-4 Secretary

SG-5 – Object-Oriented Technology

Peter Heller, Airbus Operations GmbH
Jan-Hendrik Boelens, Eurocopter

James Hunt, aicas
Jim Chelini, Verocel
Greg Millican, Honeywell

Jim Chelini, Verocel

SG-5 Co-chair (until February 2009)
SG-5 Co-chair (Feb 2009 to August 2010)
SG-5 Co-chair (from August 2010)
SG-5 Co-chair (until Oct 2009)
SG-5 Co-chair (Oct 2009 to August 2011)
SG-5 Co-chair (from August 2011)

SG-6 – Formal Methods

Duncan Brown, Aero Engine Controls (Rolls-Royce)
Kelly Hayhurst, NASA

SG-6 Co-chair
SG-6 Co-chair

SG-7 – Special Considerations and CNS/ATM

David Hawken, NATS
Jim Stewart, NATS
Don Heck, Boeing Company
Leslie Alford, Boeing Company
Marguerite Baier, Honeywell

SG-7 Co-chair (until June 2010)
SG-7 Co-chair (from June 2010)
SG-7 Co-chair
SG-7 Secretary (until March 2008)
SG-7 Secretary (from March 2008)

RTCA Representative:

Rudy Ruana
Ray Glennon
Hal Moses
Cyndy Brown
Hal Moses

RTCA Inc. (until September 2009)
RTCA Inc. (until March 2010)
RTCA Inc. (until August 2010)
RTCA Inc. (until August 2011)
RTCA Inc. (from August 2011)

EUROCAE Representative:

Gilbert Amato
Roland Mallwitz

EUROCAE (until September 2009)
EUROCAE (from October 2009)

EDITORIAL COMMITTEE

Leanna Rierson, Digital Safety Consulting
Ron Ashpole, SILVER ATENA
Alex Ayzenberg, Boeing Company
Patty (Bartels) Bath, Esterline AVISTA
Dewi Daniels, Verocel
Hervé Delseny, Airbus
Andrew Elliott, Design Assurance
Kelly Hayhurst, NASA
Barbara Lingberg, FAA
Steven C. Martz, Garmin
Steve Morton, TBV Associates
Marge Sonnek, Honeywell

Editorial Committee Chair
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee
Editorial Committee

COMMITTEE MEMBERSHIP

Name	Organization
Kyle Achenbach	Rolls-Royce
Dana E. Adkins	Kidde Aerospace
Leslie Alford	Boeing Company
Carlo Amalfitano	Certcon Software, Inc
Gilbert Amato	EUROCAE
Peter Amey	Praxis High Integrity Systems
Robert Annis	GE Aviation
Allan Gilmour Anderson	Embraer
Håkan Anderwall	Saab AB
Joseph Angelo	NovAtel Inc, Canada
John Charles Angermayer	Mitre Corp
Ron Ashpole	SILVER ATENA
Alex Ayzenberg	Boeing Company
Marguerite Baier	Honeywell
Fred Barber	Avidyne
Clay Barber	Garmin International
Gerald F. Barofsky	L-3 Communications
Patty (Bartels) Bath	Esterline AVISTA
Brigitte Bauer	Thales
Phillipe Baufreton	SAGEM DS Safran Group
Connie Beane	ENEA Embedded Technology Inc
Bernard Beaudouin	EADS APSYS
Germain Beaulieu	Independent Consultant
Martin Beeby	Seaweed Systems
Scott Beecher	Pratt & Whitney
Haik Biglari	Fairchild Controls
Peter Billing	Aviya Technologies Inc
Denise Black	Embedded Plus Engineering
Brad Blackhurst	Independent Consultant
Craig Bladow	Woodward
Martha Blankenberger	Rolls-Royce
Holger Blasum	SYSGO
Thomas Bleichner	Rohde & Schwarz
Don Bockenfeld	CMC Electronics
Jan-Hendrik Boelens	Eurocopter
Eric Bonnafous	CommunicationSys
Jean-Christophe Bonnet	CEAT
Hugues Bonnin	Cap Gemini
Matteo Bordin	AdaCore
Feliks Bortkiewicz	Boeing
Julien Bourdeau	DND (Canada)
Paul Bousquet	Volpe National Transportation Systems Center
David Bowen	EUROCAE
Elizabeth Brandli	FAA
Andrew Bridge	EASA

Name	Organization
Paul Brook	Thales
Daryl Brooke	Universal Avionics Systems Corporation
Cyndy Brown	RTCA, Inc.
Duncan Brown	Aero Engine Controls (Rolls-Royce)
Thomas Buchberger	Siemens AG
Brett Burgeles	Consultant
Bernard Buscail	Airbus
Bob Busser	Systems and Software Consortium
Christopher Caines	QinetiQ
Cristiano Campos Almeida De Freitas	Embraer
Jean-Louis Camus	Esterel Technologies
Richard Canis	EASA
Yann Carlier	DGAC
Luc Casagrande	EADS Apsys
Mark Chapman	Hamilton Sundstrand
Scott Chapman	FAA
Jim Chelini	Verocel
Daniel Chevallier	Thales
John Chilenski	Boeing Company
Subbiah Chockalingam	HCL Technologies
Chris Clark	Sysgo
Darren Cofer	Rockwell Collins
Keith Coffman	Goodrich
John Coleman	Dawson Consulting
Cyrille Comar	AdaCore
Ray Conrad	Lockheed Martin
Mirko Conrad	The MathWorks, Inc.
Nathalie Corbovianu	DGAC
Ana Costanti	Embraer
Dewi Daniels	Verocel
Eric Danielson	Rockwell Collins
Henri De La Vallée Poussin	SABCA
Michael Deitz	Gentex Corporation
Jean-Luc Delamaide	EASA
Hervé Delseny	Airbus
Patrick Desbiens	Transport Canada
Mike DeWalt	Certification Services, Inc./FAA
Mansur Dewshi	Ultra Electronics Controls
Bernard Dion	Esterel Technologies
Antonio Jose Vitorio Domiciano	Embraer
Kurt Doppelbauer	TTTech
Cheryl Dorsey	Digital Flight
Rick Dorsey	Digital Flight
John Doughty	Garmin International
Vincent Dovydaytis III	Foliage Software Systems, Inc.
Georges Duchain	DGA

Name	Organization
Branimir Dulic	Transport Canada
Gilles Dulon	SAGEM DS Safran Group
Paul Dunn	Northrop Grumman Corporation
Andrew Eaton	UK CAA
Brian Eckmann	Universal Avionics Systems Corporation
Vladimir Eliseev	Sukhoi Civil Aircraft Company (SCAC)
Andrew Elliott	Design Assurance
Mike Elliott	Boeing Company
Joao Esteves	Critical Software
Rowland Evans	Pratt & Whitney Canada
Louis Fabre	Eurocopter
Martin Fassl	Siemens AG
Michael Fee	Aero Engine Controls (Rolls-Royce)
Tom Ferrell	Ferrell and Associates Consulting
Uma Ferrell	Ferrell and Associates Consulting
Lou Fisk	GE Aviation
Ade Fountain	Penny and Giles
Claude Fournier	Liebherr
Pierre Francine	Thales
Timothy Frey	Honeywell
Stephen J. Fridrick	GE Aviation
Leonard Fulcher	TTTech
Randall Fulton	Seaweed Systems
Francoise Gachet	Dassault-Aviation
Victor Galushkin	GosNIIAS
Marty Gasirowski	Worldwide Certification Services
Stephanie Gaudan	Thales
Jean-Louis Gebel	Airbus
Dries Geldof	BARCO
Dimitri Giancesini	Airbus
Jim Gibbons	Boeing Company
Dara Gibson	FAA
Greg Gicca	AdaCore
Steven Gitelis	Lumina Engineering
Ian Glazebrook	WS Atkins
Santiago Golmayo	GMV SA
Ben Gorry	British Aerospace Systems
Florian Gouleau	DGA Techniques Aéronautiques
Olivier Graff	Intertechnique - Zodiac
Russell DeLoy Graham	Garmin International
Robert Green	BAE Systems
Mark Grindle	Systems Enginuity
Peter Grossinger	Pilatus Aircraft
Mark Gulick	Solers, Inc.
Pierre Guyot	Dassault Aviation
Ibrahim Habli	University of York
Ross Hannan	Sigma Associates (Aerospace) Limited

Name	Organization
Christopher H. Hansen	Rockwell Collins
Wue Hao Wen	Civil Aviation Administration of China (CAAC)
Keith Harrison	HVR Consulting Services Ltd
Bjorn Hasselqvist	Saab AB
Kevin Hathaway	Aero Engine Controls (Goodrich)
David Hawken	NATS
Kelly Hayhurst	NASA
Peter Heath	Securaplane Technologies
Myron Hecht	Aerospace Corporation
Don Heck	Boeing Company
Peter Heller	Airbus Operations GmbH
Barry Hendrix	Lockheed Martin
Michael Hennell	LDRA
Michael Herring	Rockwell Collins
Ruth Hirt	FAA
Kent Hollinger	Mitre Corp
C. Michael Holloway	NASA
Ian Hopkins	Aero Engine Controls (Rolls-Royce)
Gary Horan	FAA
Chris Hote	PolySpace Inc.
Susan Houston	FAA
James Hummell	Embedded Plus
Dr. James J. Hunt	aicas
Rebecca L. Hunt	Boeing Company
Stuart Hutchesson	Aero Engine Controls (Rolls-Royce)
Rex Hyde	Moog Inc. Aircraft Group
Mario Iacobelli	Mannarino Systems
Melissa Isaacs	FAA
Vladimir Istomin	Sukhoi Civil Aircraft Company (SCAC)
Stephen A. Jacklin	NASA
Matt Jaffe	Embry-Riddle Aeronautical University
Marek Jaglarz	Pilatus Aircraft
Myles Jalalian	FAA
Merlin James	Garmin International
Tomas Jansson	Saab AB
Eric Jenn	Thales
Lars Johannknecht	EADS
Rikard Johansson	Saab AB
John Jorgensen	Universal Avionics Systems
Jeffrey Joyce	Critical Systems Labs
Chris Karis	Ensco
Gene Kelly	CertTech
Anne-Cécile Kerbrat	Aeroconseil
Randy Key	FAA
Charles W. Kilgore II	FAA
Wayne King	Honeywell
Daniel Kinney	Boeing Company

Name	Organization
Judith Klein	Lockheed Martin
Joachim Klichert	Diehl Avionik Systeme
Jeff Knickerbocker	Sunrise Certification & Consulting, Inc.
John Knight	University of Virginia
Rainer Kollner	Verocel
Andrew Kornecki	Embry-Riddle Aeronautical University
Igor Koverninskiy	Gos NIIAS
Jim Krodel	Pratt & Whitney
Paramesh Kunda	Pratt & Whitney Canada
Sylvie Lacabanne	AIRBUS
Gérard Ladier	Airbus/Aerospace Valley
Ron Lambalot	Boeing Company
Boris Langer	Diehl Aerospace
Susanne Lanzerstorfer	APAC GesmbH
Gilles Laplane	SAGEM DS Safran Group
Jeanne Larsen	Hamilton Sundstrand
Emmanuel Ledinet	Dassault Aviation
Stephane Leriche	Thales
Hong Leung	Bell Helicopter Textron
John Lewis	FAA
John Li	Thales
Mark Lillis	Goodrich GPECS
Barbara Lingberg	FAA
Pierre Lionne	EADS APSYS
Hoyt Lougee	Foliage Software Systems
Howard Lowe	GE Aviation
Hauke Luethje	NewTec GmbH
Jonathan Lynch	Honeywell
Françoise Magliozzi	Atos Origin
Veronique Magnier	EASA
Kristine Maine	Aerospace Corporation
Didier Malescot	DSNA/DTI
Varun Malik	Hamilton Sundstrand
Patrick Mana	EUROCONTROL
Joseph Mangan	Coanda Aerospace Software
Ghilaine Martinez	DGA Techniques Aéronautiques
Steven C. Martz	Garmin International
Peter Matthews	Independent Consultant
Frank McCormick	Certification Services Inc
Scott McCoy	Harris Corporation
Thomas McHugh	FAA
William McMin	Lockheed Martin
Josh McNeil	US Army AMCOM SED
Kevin Meier	Cessna Aircraft Company
Amanda Melles	Bombardier
Marc Meltzer	Belcan Engineering
Steven Miller	Rockwell Collins

Name	Organization
Gregory Millican	Honeywell
John Minihan	Resource Group
Martin Momberg	Cassidian Air Systems
Pippa Moore	UK CAA
Emilio Mora-Castro	EASA
Endrich Moritz	Technical University
Robert Morris	CDL Systems Ltd.
Allan Terry Morris	NASA
Steve Morton	TBV Associates
Harold Moses	RTCA, Inc.
Nadir Mostefat	Mannarino Systems
Fred B. Moyer	Rockwell Collins
Robert D. Mumme	Embedded Plus Engineering
Arun Murthi	AERO&SPACE USA
Armen Nahapetian	Teledyne Controls
Gerry Ngu	EASA
Elisabeth Nguyen	Aerospace Corporation
Robert Noel	Mitre Corp
Sven Nordhoff	SQS AG
Paula Obeid	Embedded Plus Engineering
Eric Oberle	Becker Avionics
Brenda Ocker	FAA
Torsten Ostermeier	Bundeswehr
Frederic Painchaud	Defence Research and Development Canada
Sean Parkinson	Resource Group
Dennis Patrick Penza	AVISTA
Jean-Phillipe Perrot	Turbomeca
Robin Perry	GE Aviation
David Petesch	Hamilton Sundstrand
John Philbin	Northrop Grumman Integrated Systems
Christophe Piala	Thales Avionics
Cyril Picard	EADS APSYS
Francine Pierre	Thales Avionics
Patrick Pierre	Thales Avionics
Gerald Pilj	FAA
Benoit Pinta	Intertechnique - Zodiac
Mo Piper	Boeing Company
Andreas Pistek	ITK Engineering AG
Laurent Plateaux	DGA
Laurent Pomies	Independent Consultant
Jennifer Popovich	Jeppesen Inc.
Clifford Porter	Aircell LLC
Frédéric Pothon	ACG Solutions
Bill Potter	The MathWorks Inc
Sunil Prasad	HCL Technologies, Chennai, India
Paul J. Prisaznuk	ARINC-AEEC
Naim Rahmani	Transport Canada

Name	Organization
Angela Rapaccini	ENAC
Lucas Redding	Silver-Atena
David Redman	Aerospace Vehicle Systems Institute (AVSI)
Tammy Reeve	Patmos Engineering Services, Inc.
Guy Renault	SAGEM DS Safran Group
Leanna Rierson	Digital Safety Consulting
George Romanski	Verocel
Cyrille Rosay	EASA
Edward Rosenbloom	Kollsman, Inc
Tom Roth	Airborne Software Certification Consulting
Jamel Rouahi	CEAT
Marielle Roux	Rockwell Collins France
Benedito Massayuki Sakugawa	ANAC Brazil
Almudena Sanchez	GMV SA
Vdot Santhanam	Boeing Company
Laurence Scales	Thales
Deidre Schilling	Hamilton Sundstrand
Ernst Schmidt	Bundeswehr
Peter Schmitt	Universität Karlsruhe
Dr. Achim Schoenhoff	EADS Military Aircraft
Martin Schwarz	TT Technologies
Gabriel Scolan	SAGEM DS Safran Group
Christel Seguin	ONERA
Beatrice Sereno	Teuchos SAFRAN
Phillip L. Shaffer	GE Aviation
Jagdish Shah	Parker
Vadim Shapiro	TetraTech/AMT
Jean François Sicard	DGA Techniques Aéronautiques
Marten Sjoestedt	Saab AB
Peter Skaves	FAA
Greg Slater	Rockwell Collins
Claudine Sokoloff	Atos Origin
Marge Sonnek	Honeywell
Guillaume Soudain	EASA
Roger Souter	FAA
Robin L. Sova	FAA
Richard Spencer	FAA
Thomas Sperling	The Mathworks
William StClair	LDRA
Roland Stalford	Galileo Industries Spa
Jerry Stamatopoulos	Aircell LLC
Tom Starnes	Cessna Aircraft Company
Jim Stewart	NATS
Tim Stockton	Certon
Victor Strachan	Northrop-Grumman
John Strasburger	FAA
Margarita Strelnikova	Sukhoi Civil Aircraft Company (SCAC)

Name	Organization
Ronald Stroup	FAA
Will Struck	FAA
Wladimir Terzic	SAGEM DS Safran Group
Wolfgang Theurer	C-S SI
Joel Thornton	Tier5 Inc
Mikael Thorvaldsson	KnowIT Technology
Bozena Brygida Thrower	Hamilton Sundstrand
Christophe Travers	Dassault Aviation
Fay Trowbridge	Honeywell
Nick Tudor	Tudor Associates
Silpa Uppalapati	FAA
Marie-Line Valentin	Airbus
Jozef Van Baal	Civil Aviation Authorities Netherlands
John Van Leeuwen	Sikorsky Aircraft
Aulis Viik	NAV Canada
Bertrand Voisin	Dassault Aviation
Katherine Volk	L-3 Communications
Dennis Wallace	FAA
Andy Wallington	Bell Helicopter
Yunming Wang	Esterel Technologies
Don Ward	AVSI
Steve Ward	Rockwell Collins
Patricia Warner	Software Engineering
Michael Warren	Rockwell Collins
Rob Weaver	NATS
Yu Wei	CAA China
Terri Weinstein	Parker Hannifin
Marcus Weiskirchner	EADS Military Aircraft
Daniel Weisz	Sandel Avionics, Inc.
Rich Wendlandt	Quantum3D
Michael Whalen	Rockwell Collins
Paul Whiston	High Integrity Solutions Ltd
Todd R. White	L-3 Communications/Qualtech
Virginie Wiels	ONERA
ElRoy Wiens	Cessna Aircraft Company
Terrance Williamson	Jeppesen Inc.
Graham Wisdom	BAE Systems
Patricia Wojnarowski	Boeing Commercial Airplanes
Joerg Wolfrum	Diehl Aerospace
Kurt Woodham	NASA
Cai Yong	CAAC (Civil Aviation Administration of China)
Edward Yoon	Curtiss-Wright Controls, Inc
Robert Young	Rolls-Royce
William Yu	CAAC China
Erhan Yuceer	Savunma Teknolojileri Muhendislik ve Ticaret
Uli Zanker	Liebherr

APPENDIX FM.B FREQUENTLY ASKED QUESTIONS AND DISCUSSION PAPERS

This appendix contains a discussion paper about using formal methods with DO-178C.

FM.B.1 DP #1: EXAMPLES OF FORMAL METHODS

Reference: FM.1; FM.6

Key words: formal methods; verification; proof; model checking

FM.B.1.1 Background Information

During the process of creating the Formal Methods Supplement, many questions were asked about how formal methods are used in practice in the aerospace industry. The Formal Methods Supplement is intentionally restricted to discussion of certification guidance. This discussion paper provides background information and several examples of actual use of formal methods on real projects. It describes how the objectives in the Formal Methods Supplement could be satisfied using various technologies.

FM.B.1.2 Introduction

Formal methods are mathematically based techniques for the specification, development and verification of computer systems. The use of formal methods is motivated by the fact that performing rigorous mathematical analyses can contribute to establishing the correctness and robustness of software aspects of safety-critical systems. Such techniques are also highly applicable to complex hardware development assurance. The formal logic, discrete mathematics, and computer-readable languages underpinning formal methods, provide a solid and defensible foundation for many of the development and verification activities required for avionics software. Nonetheless, the avionics industry at large has been hesitant to adopt formal methods, despite growing evidence of their benefits.

Since the introduction of formal methods as an alternative method in section 12.3.1 of DO-178B, advances and practical experience have been gained in techniques and tools supporting formal methods, to the extent that they have become sufficiently mature for application on today's avionics products. The Formal Methods Supplement was written, consequently, to provide guidance for applicants and certification authorities to facilitate the use of formal methods. Practicable information, however, has not been readily accessible to those in the avionics industry without specialized training in formal methods. The field of formal methods is quite broad, encompassing many types of mathematical theory, specialized terminology, notations, and symbols, and numerous analysis approaches and tools. This can be daunting to non-specialists and contribute to a slow up-take of this technology.

This discussion paper is a first step toward bridging the information gap, for the express purpose of augmenting the information in this supplement. The text of this supplement deliberately avoids formal methods jargon and identification of specific approaches. That is, this supplement provides guidance to facilitate the general use of formal methods to meet DO-178 objectives, without explicitly describing how to use particular methods to meet the objectives. This paper provides a glimpse at the "how to" for non-specialists. To that end, basic concepts and terminology for formal methods are presented, along with examples to illustrate how formal methods have been used to meet DO-178 objectives, and benefits for doing so. This paper does not form a definitive guide of the only valid

ways of applying formal methods and is not an exhaustive list of approaches, tools or techniques.

Section FM.B.1.3 of this discussion paper explains fundamental formal methods concepts and terminology for modeling and analysis, and gives an overview of formal languages and tools. Section FM.B.1.4 describes some general benefits of formal methods; while section FM.B.1.5 gives some simple examples from the aerospace domain of how formal methods can be used to meet the DO-178 objectives, as described in this supplement. Finally, section FM.B.1.6 summarizes the extent to which formal methods can be applied to DO-178 compliance.

FM.B.1.3 What Are Formal Methods?

Formal methods are mathematically based techniques for the specification, development and verification of software aspects of digital systems. The mathematical basis of formal methods consists of formal logic, discrete mathematics, and computer-readable languages. The use of formal methods is motivated by the expectation that, as in other engineering disciplines, performing appropriate mathematical analyses can contribute to establishing the correctness and robustness of a design. For example, formal methods, because of their mathematical basis, are capable of:

- unambiguously describing requirements of software systems.
- enabling precise communication between engineers.
- providing objective verification evidence such as consistency and accuracy of a formally specified representation of software.
- providing objective verification evidence of the compliance of one formally specified representation with another.

Formal methods are also capable of demonstrating aspects of software systems, generally termed properties when using formal methods, like:

- freedom from exceptions.
- freedom from deadlock.
- non-interference between different levels of criticality.
- worst-case execution timing.
- bounds on stack size during execution.
- freedom from unintended function.
- correct synchronous or asynchronous behavior.

Understanding some of the fundamental characteristics of formal methods and their scope of applicability is helpful for understanding the examples in section FM.B.1.5. Application of formal methods usually involves two types of activities: modeling and

analysis, where the models and analysis procedures are derived from or defined by an underlying mathematically precise foundation. Although modeling and analysis are common activities in many software engineering efforts, these terms have particular meanings when used in the context of formal methods described in this document. Not all models nor all analyses will meet requirements for being formal. A formal model combined with formal analysis constitute a formal method.

FM.B.1.3.1 Formal Models

Establishing a formal model of the software artifact of interest is fundamental to all formal methods. In general, a model is an abstract representation of a given set of aspects of the software that is used for analysis, simulation and/or code generation. In the context of this document, to be formal, a model must have an unambiguous, mathematically defined syntax and semantics. This makes it possible to use automated means to obtain guarantees that the model has certain specified properties.

Formal models can be produced by the development process (for example, requirement or design activities) or by analysis of the software artifact. Creating a formal model by analyzing a software artifact is possible when the artifact itself exhibits the characteristics of well defined syntax and semantics; and therefore, for the purposes of this document, artifacts such as source and object code can also be thought of as formal models in these cases.

Throughout the development life cycle, different formal models may be used for different kinds of analysis and for the establishment of different properties. Examples of formal models include:

- Graphical models where the components of the diagram and the connections between them have mathematically defined syntax and semantics; for example, control diagrams or state machines.
- Textual based models such as Z, logic, set theory, algebraic or differential equations or even a subset of C or Ada that are selected to have a mathematically defined syntax and semantics.
- Abstract models formed inside a formal methods tool by an analysis of the software artifacts of interest such as the code.

The applicability of formal methods to any particular software development activity is bounded by the ability to construct an appropriate formal model. A single model, or even a combination of models, may not be sufficient to capture all of the aspects of the software that are required to be verified according to the objectives of DO-178. That is, in many cases, it is not possible to completely model the software using a formal notation. Requirements specified in natural language (such as English) may include properties that cannot be reasoned about with a formal method. Models can also be insufficiently detailed to allow meaningful analysis of some properties and yet perfectly adequate for others.

Despite these limitations, formal models can be a means of completely and accurately describing important software properties. Formal analyses can then be applied to provide assurance of these properties and combined with other verification techniques (for example, testing) to achieve satisfaction of all of the verification objectives. Formal

methods provide comprehensive assurance of particular properties only for those aspects that are formalized in the formal model, so defining the limits of the model is essential.

Formal methods may be applied in many stages of the development life cycle, and to satisfy various verification objectives. Wherever formal models are used, they should be verified to provide assurance that they are an accurate representation of the artifact. This means that for the properties being analyzed, if they hold for the model then they hold for the actual artifact. This is easier to demonstrate if the gap between an informal statement of a requirement and its embodiment in a formal notation is not too large.

FM.B.1.3.2 Formal Analysis

Although there are important benefits in creating formal models of life cycle artifacts, the most powerful benefits of formal methods are in the formal analysis of those models. Formal analysis can provide guarantees or proofs of software properties and compliance with requirements. Proof, or guarantee, implies that all execution cases are taken into account. To conduct a formal analysis a set of software properties is necessary. These properties are either created or they are embedded in specific tools that implement the formal analysis (for example, worst-case execution timing). Properties for formal analysis can be seen as analogous in many ways to verifiable requirements for testing. In testing, test cases should be easily derivable from verifiable requirements, for formal analysis the properties are usually also the verification cases. It may be possible to define a complete set of properties for a set of requirements. In this case proof that the set of properties always holds true demonstrates the equivalence of the item under analysis to its requirements.

For the purpose of this document, an analysis method can only be regarded as formal analysis if its determination of a property is sound. Sound analysis means that the method never asserts a property to be true when it may not be true. The converse case, the assertion that a property is false when it may be true, colloquially “the raising of false alarms”, is a usability issue but not a soundness issue. Furthermore, it is acceptable for a method to return “don’t know” or not to return an answer when trying to establish whether a property holds, in which case additional verification is necessary.

There are many different kinds of formal analysis, but they can typically be classified in three categories: (1) deductive methods, such as theorem proving, (2) model checking, and (3) abstract interpretation.

Deductive methods involve mathematical arguments, such as mathematical proofs, for establishing a specified property of a formal model. A correct proof of a property provides rigorous evidence of that property for the formal model. These proofs are typically constructed using an automated or interactive theorem proving system. Even with such assistance, constructing proofs can be difficult, or impossible in some cases. However, once a proof is completed, automatic checking of the correctness of that proof is usually trivial.

Model checking explores all possible behaviors of a formal model to determine whether a specified property is satisfied. In general a model check can have three outcomes. If the model checker determines that all the specified properties are satisfied, it will simply give an affirmative answer. In cases where the property is not satisfied, a counter-example illustrating where and how the property fails to hold is generated automatically. In some cases, a model checker may not be able to determine whether the given property is satisfied or not.

Abstract Interpretation is a theory for formally constructing conservative representations (i.e. enforcing soundness) of the semantics of programming languages. In practice, this method is used for constructing semantics-based analysis algorithms for the automatic, static and sound determination of dynamic properties of infinite-state programs. With abstract interpretation, a formal model is generated, usually within a tool, which is specific to the particular property under analysis. It can be viewed as a partial execution of a computer program which determines specific effects of the program (for example, control structure, flow of information, stack size, number of clock cycles) without actually performing all the calculations.

All of these formal analysis methods share two important attributes. First, the methods rely on formal models. Compliance between artifacts can never be demonstrated between an informal model and a formal model, using formal analysis. For example, using formal methods to demonstrate compliance between specification and code requires that both be formal models. Second, all of the formal analyses are generally implemented using a tool. Any tool that supports the formal analysis should be assessed under the tool qualification guidance required by DO-178 and qualified where necessary.

FM.B.1.3.3 Formal Methods as a Verification Technique

Formal methods may enable complete exploration of possible behaviors of a formal model with respect to certain properties. Subject to the formal model employed being an accurate representation of the artifact (which should be established by additional verification) formal methods provide assurance that certain kinds of faults are not present at the level of description and stage of the life cycle considered by the formal model. This document provides examples for meeting some of the verification objectives of DO-178. All the objectives that are not mentioned here are achieved using other methods. Functional tests executed in target hardware are always required to establish correctness of the overall system.

FM.B.1.4 What Can They Do For Me?

Formal methods have the potential for both increasing safety and decreasing the effort of certifying flight-critical systems. Firstly, experience shows that the very act of capturing requirements using formal notations is of benefit: it forces the writer to ask all sorts of questions that would otherwise be postponed until coding. Secondly, the formality of the description allows us to carry out rigorous analyses. Such analyses can verify useful properties such as consistency, deadlock-freedom, and satisfaction of high-level requirements or correctness of a proposed design.

FM.B.1.4.1 Improve Requirements

It is more effective to write requirements formally than to write an informal one. Such requirements provide a simple validation check, as it forces a level of explicitness far beyond that needed for informal representations. They can also be analyzed early to detect inconsistency and incompleteness and therefore remove errors that would normally not be found until later in the process.

FM.B.1.4.2 Reduce Error Introduction

Writing requirements formally (high or low level) improves the quality of the development artifacts. Formalized requirements compared to informal ones prevent misunderstandings which lead to error introduction. As development proceeds,

compliance can be continually checked using a formal analysis to detect errors that may have been introduced.

FM.B.1.4.3 Improve Error Detection

Formal analysis can provide exhaustive verification of software at whatever levels it is applied: high-level requirements, low-level requirements, Source Code or executable. Exhaustive verification means that all of the structure is verified over all possible inputs and states. This can detect errors that would be difficult or impossible to find using only a test based approach.

FM.B.1.4.4 Reduce Effort

Since formal methods can be used to discover errors early in the life cycle, they actually reduce the overall work required for the project. Errors are easier to correct the earlier they are detected. When requirements have been formalized the effort of downstream activities is reduced. Formal notations also reduce effort by enabling the automation of verification activities.

FM.B.1.4.5 Supporting Data

Experimental data supporting the use of formal methods can be found in a case study sponsored by a government agency [Wha07, Mil09]. In this study, model checking and test-based verification techniques were evaluated through application to control software of an experimental unmanned air vehicle (UAV). The study was not conducted to DO-178 standards but provides evidence regarding effectiveness and efficiency of formal methods versus testing.

To this end, two teams were set up to verify the redundancy management components of the UAV control system. One team used traditional testing of the software. The other team performed verification using model checking tools. Neither team communicated directly with the other, and both teams started with identical software models and requirements to be verified.

Both teams developed extensions to their base verification technologies. The model-checking team extended their existing tools to add support for several new block types found in the UAV software. Likewise, the testing team also made comparable investments in enhancing their testing environment. These one-time, non-recurring costs were not included in the final comparison of the effectiveness of testing and model checking.

The redundancy management software is organized into three components that could be analyzed individually (see Table FM.B.1-1).

Table FM.B.1-1 Redundancy Management Software Verification Results

Component	Subsystem Instances	Basic Blocks	Number of Properties	Errors found by Model Checking	Errors found by Testing
Triplex Voter	10	96	43	5	0
Failure Processing	7	42	6	3	0
Reset Manager	6	31	8	4	0
Totals	23	169	62	12	0

The model checking team developed a total of 62 properties for analysis from the original software requirements. Analysis of these properties with the model checker uncovered 12 errors in the redundancy management logic. In similar fashion, the testing team developed a series of tests cases from the same set of software requirements. Even though the testing team invested nearly 50% more time in testing than the formal verification team spent in model checking, testing failed to find any errors in the software. The conclusion of both teams was that in this case study, model checking was more effective and more efficient than testing in finding design errors. Some of the errors found by the model checking team would be difficult, if not impossible, to discover through testing. For example one such error involved a complex timing interaction between the inputs to the Triplex Voter which resulted in a good sensor being declared faulty.

This case study provides a concrete demonstration of the fact that model checking can be more effective at finding certain errors than testing, which can only check a small fraction of the possible inputs and states. Testing would normally be used in conjunction with model checking as there are classes of errors which model checking is unlikely to find. Furthermore, this case study demonstrates that model checking can be accomplished with less effort than testing in some cases.

FM.B.1.5 Examples

The adoption of formal methods into an established set of processes for development and verification can be an evolutionary refinement rather than an abrupt change of methodology. The extent to which formal methods are used in a particular project to satisfy the objectives of DO-178 can vary according to various aspects such as the preferences of the program management, the choice of techniques and the availability of specialized resources. Formal methods might be used in a very selective manner to partially address a small set of objectives, or might be the primary source of evidence for the satisfaction of many of the objectives concerned with development and verification.

The following are examples of what can be achieved with formal methods, but are not a complete set of valid approaches. Each example shows how certification evidence can be generated. This should not be taken to be complete. It cannot be assumed that a certification authority will accept the evidence in every context.

The example in FM.B.1.5.1 addresses unit proof of Source Code, and the example in FM.B.1.5.2 addresses model checking compliance of low-level requirements (LLR) against high-level requirements (HLR).

FM.B.1.5.1 Example 1: Unit Proof

In this example, a development cycle with the following characteristics is considered:

- HLR are informally expressed.
- LLR are formal properties expressed in a first order logic.
- Unit verification is done by formal proof.
- Integration verification is done by test.

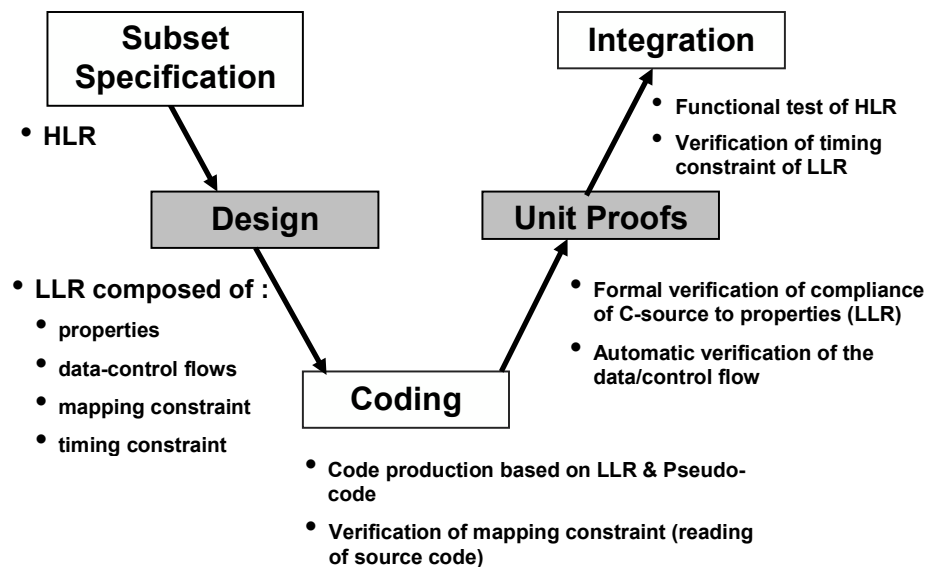


Figure FM.B.1-1 Software Life Cycle for Unit Proof

In this example, objectives are illustrated on a concrete (simple) example, the check of a flash zone. In the following, the HLR, LLR and Source Code for this example are given, then how the objectives of this supplement can be achieved is explained. More information can be found in [Fam99] and [Sou09].

HLR

The High Level Requirement we are considering is the following:

“Check of a flash zone:

- verify that the whole flash zone is initialized to value 0xFF,
- if a memory location is different from 0xFF, the check has failed.”

LLR

LLR are composed of a textual description and a set of formal properties.

Textual description

This service implements an integrity check of flash zone by checking that the whole flash zone is initialized to 0xFF.

Return value

- check result

Parameters

Input:

- none

Output:

- pPanne: failure description

Input/output:

- none

Declaration

A1F2_TestZone(pPanne : out T_DESCRIPTEUR_PANNE) return T_RESULTAT;

Formal Properties

- Formal property language

In this example, formal properties are expressed in first order logic, because this is the language required for using the theorem proving tool. The logical operators used are the following:

$\wedge$: AND

$\vee$: OR

$\neg$: NOT

$\Rightarrow$: IMPLIES (A $\Rightarrow$ B means “if A then B”)

In addition to these operators, quantifiers $\exists$ (there exists) and $\forall$ (for all) are used to express properties involving indexes of the memory zone, the formula:

$$\exists k \in \text{int. } k > 0 \wedge k \leq n \wedge \text{Prop}(k)$$

expresses the fact that there exist an index k (that is an integer) between 1 and n such that the property Prop is verified for this index k. The formula

$$\forall k \in \text{int. } k > 0 \wedge k \leq n \Rightarrow \text{Prop}(k)$$

expresses the fact that for all the indexes from 1 to n, the property Prop is verified.

- Properties for the example

The properties are decomposed into five parts. Part 1 defines all the execution conditions of the service, the execution conditions are of three kinds: functional conditions, robustness conditions and failure conditions. These execution conditions are used in Part 5. Part 2 defines some properties to check the completeness and consistency of the execution conditions. This part thus does not contain properties on the code itself, but properties on the execution conditions of the code. The objective is to be sure that no execution conditions are forgotten (completeness) and that the conditions that have been defined are mutually exclusive (consistency). In this small example, COND-ERR is the negation of COND-FCT, so the verification of properties of part 2 is easy, but it is not always the case for more complex examples. Part 3 defines the possible values for the inputs. Part 4 defines the possible values for the outputs. Part 5 specifies the value of the outputs of the service for each execution condition; this is the part that specifies the expected behavior of the code.

-- 1- Definitions of execution conditions

-- Definition of functional conditions

LET COND_FCT = (
 $\forall k \in \text{int. } k > 0 \wedge k \leq \text{A1F2_ZONE_SIZE} \Rightarrow (\text{A1F2_Memory_Zone}[(k)] = 0xFF));$
The initial value is correct for all the indexes

-- Definition of robustness conditions

-- Not required in this example because the memory test either results in “OK” or “NOT_OK”

-- Definition of failure conditions

LET COND_ERR = (
 $\exists k \in \text{int. } k > 0 \wedge k \leq \text{A1F2_ZONE_SIZE} \wedge (\text{A1F2_Memory_Zone}[(k)] \neq 0xFF));$
There exists an index for which the initial value is wrong

-- 2- Verification of execution conditions

-- Completeness of the conditions

Always COND0: $\text{COND_ERR} \vee \text{COND_FCT};$
There is no other case apart from these two

-- Disjunction of execution conditions

Always COND_DIS1: $\neg (\text{COND_FCT} \wedge \text{COND_ERR});$
There is no case where both of these hold true

-- 3- Verification of inputs domain

-- N/A

-- 4- Verification of outputs domain

-- Return value

Post VAR1_Retour: $(A1F2_TestZone = OK) \vee (A1F2_TestZone = NOT_OK)$;
Return value is either OK or NOT_OK

-- field IndexPanne of pPanne

Post VAR2_IndexPanne :

pPanne.[*,IndexPanne] = INDEX_NO_ERROR

$\vee$ pPanne.[*,IndexPanne] = INDEX_FLASH_2 ;

Failure description is either No error or Identifier of error

-- 5- Verification of the results

-- Check result

Post RES1a: $COND_FCT \Rightarrow A1F2_TestZone = OK$;

Post RES1b: $COND_ERR \Rightarrow A1F2_TestZone = NOT_OK$;

In functional condition, the result returned by the function is OK. In case of error, the result is NOT_OK.

-- field IndexPanne of pPanne

Post RES2a_INDEX: $COND_FCT \Rightarrow pPanne.[*,IndexPanne] = INDEX_NO_ERROR$;

Post RES2b_INDEX: $COND_ERR \Rightarrow pPanne.[*,IndexPanne] = INDEX_FLASH_2$;

In functional condition, the failure description indicates there is no error. In case of error, it gives the identifier of error.

Source Code

Below, the Source Code is given on which the verification of the properties will be done. The function given here is part of a software that is activated at the initialization of the computer, the role of this software is to check that the correct operation of the hardware. The function is called A1F2_TestZone, it checks a memory zone, the return value gives the result of the check and a failure description is updated. T-DESCRIPTEUR_PANNE is a record containing several fields that are used to store information about the type of failure that occurred.

```
/*-----
* Service A1F2_TestZone
* -----*/
T_RESULTAT A1F2_TestZone (T_DESCRIPTEUR_PANNE * const /* out */ pPanne)
{
    T_RESULTAT Rl_Return ;
    T_WORD32 Rl_Expectedvalue ;
    T_INT32 Rl_Index ;

    /*$ -- Init of out parameters */
    *pPanne = APAT_Ce_sPANNE_NCD ;

    /*$ -- Return value of the service */
    Rl_Return = OK ;

    /*$ -- Variable Rl_Expectedvalue of type T_WORD32 */
    Rl_Expectedvalue = 0xFF ;
```

```
/*$ -- Treatment */
Rl_Index=0;
while ((Rl_Index<=A1F2_ZONE_SIZE) && (Rl_Return==OK))
{
    if ( A1F2_Memory_Zone[Rl_Index] != Rl_Expectedvalue )
    {
        /*$ -- Failure of Flash Test */
        Rl_Return = NOT_OK ;
    }
    else
    {
        /*$ -- nothing */
    }
    /*$ -- Incrementation */
    Rl_Index = Rl_Index + 1 ;
}

/*$ -- If the whole zone is at Rl_Expectedvalue */
if (Rl_Return == OK)
{
    /*$ -- Test OK */
    /*$ -- Update of pPanne */
    *pPanne = APAT_Ce_sPANNE_NO_ERROR ;
}
else
{
    /*$ -- init of error zone */
    /*$ -- update of pPanne */
    pPanne->IndexPanne = INDEX_FLASH_2 ;
    pPanne->Vcliche2 = (T_WORD32)(A1F2_Memory_Zone[Rl_Index-1]) ;
    pPanne->Vcliche3 = 0 ;
    pPanne->Vcliche4 = Rl_Expectedvalue ;
    pPanne->Vcliche5 = A1F2_Memory_Zone[Rl_Index-1] ;
}
return Rl_Return ;
}
```

Using the Theorem Proving Tool

The following diagram synthesizes the different steps of the use of the theorem proving tool.

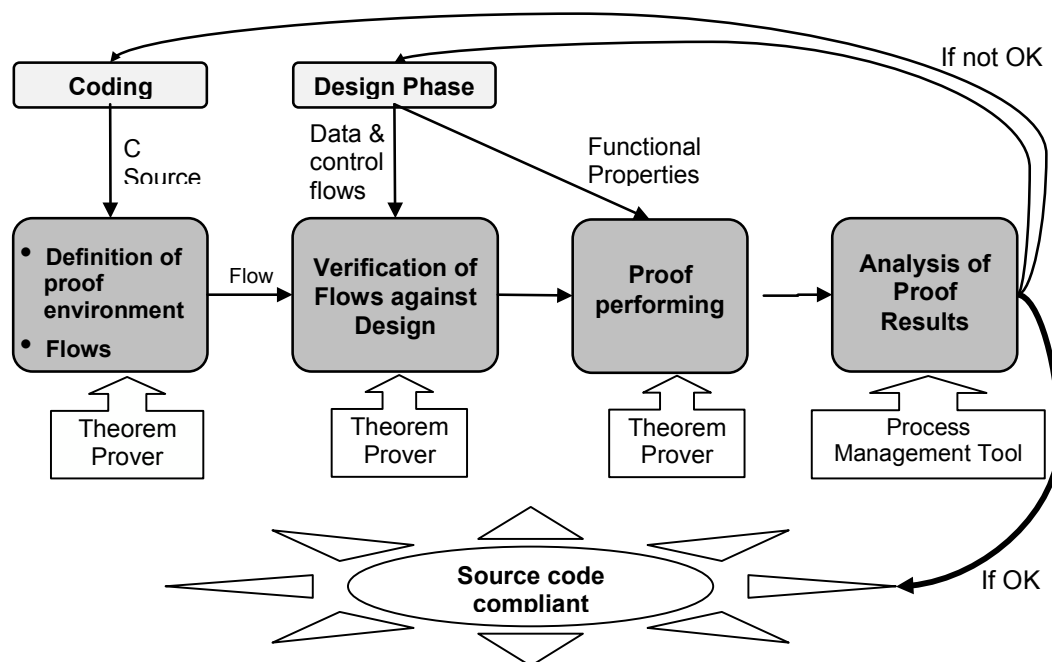


Figure FM.B.1-2 Proof Process

The proof environment is first defined and the data and control flows are automatically generated from the C code by the theorem proving tool. These flows are then verified against the data and control flows defined during the design phase. The third step is the proof of the functional properties defined during the design phase against the C code; finally, the proof results are analyzed, it must be noted that the theorem proving tool is integrated into the process management tool to automate the proof process.

Proof Performing

A brief overview of the principle of the proof (third step) is given here. The properties given above on the behavior of the function are properties that must be true after the execution of the function. To prove that these properties are verified by the code, the theorem proving tool proceeds in two steps:

- First the tool computes the conditions that must be satisfied before the execution of the program in order to ensure the satisfaction of the properties after the execution. For example, if the program is the assignment $x := x + 1$ and the property to be verified is $x = 2$, the tool will deduce that the condition that must be satisfied before the execution is $x = 1$. Rules are defined for every possible instruction of the program, and the computation of the conditions goes from the end of the program to the beginning.
- The computed conditions must then be proved using a theorem prover.

Certification Objectives

How the objectives of the formal methods technical supplement are achieved is detailed in the following. All the objectives that are not mentioned here are achieved using other

methods. In particular, functional tests executed in target hardware are always required to establish correctness of the overall system.

Table FM.A-4, Objective FM 17: Formal method is correctly defined, justified, and appropriate

And Table FM.A-5, Objective FM 13: Formal method is correctly defined, justified, and appropriate

And Table FM.A-7, Objective FM 10: Formal method is correctly defined, justified, and appropriate

- FM.6.2.1.a. Formal notations

The syntax and semantics of the subset of the C programming language are mathematically defined and verified to be precise and unambiguous. Also, the syntax and semantics of the notation used for the properties are mathematically defined and verified to be precise and unambiguous.

- FM.6.2.1.b. Soundness

The verification technique used in the theorem proving tool is based on Hoare logic that has been proved sound provided that the logic used for properties is sound. [Hoa69]

This is the case for first order logic. [Hil50]

- FM.6.2.1.c. Assumptions

No specific assumption is used for the verification of the properties on the source code.

Table FM.A-4, Objective FM 16: Requirements formalization is correct

- FM.6.3.i. Requirement formalization correctness

LLR are composed of a textual description and a set of formal properties. A review of the formal properties with respect to the textual description is performed.

Table FM.A-4, Objective 2: Low-level requirements are accurate and consistent

- FM.6.3.2.b. Accuracy and consistency

LLR are formally expressed, thus, they are unambiguous and precise in the sense that the semantics of the formal language provides an unambiguous meaning for each LLR. It does not mean that there is only one possible implementation to be compliant with the LLR. If it is necessary to ensure a specific implementation, this additional uniqueness property should be formally analyzed.

In this example, the additional analysis would consist of formally verifying that for any memory location the LLR constrains the returned status to only be one possible value. It is not the case with the LLR of this example since the procedure may return

any of a number of faulty addresses and still be compliant with the LLR. But the HLR just requires the check to fail if a memory location is different from 0xFF therefore there is no need for the uniqueness property to be proved.

Formal properties are defined and proved on the LLR using the theorem proving tool that contributes to the DO-178 consistency objective. For instance, a property is defined to verify the disjunction of the execution conditions:

-- Disjunction of execution conditions
Always COND_DIS1: $\neg (\text{COND_FCT} \wedge \text{COND_ERR})$;
There is no case where both of these hold true

This property is formally verified by the theorem proving tool.

Table FM.A-4, Objective 4: Low-level requirements are verifiable

- FM.6.3.2.d. Verifiability

The properties are expressed formally using the theorem proving tool property language and following a specific method defined for the use of the theorem proving tool, consequently they are verifiable.

Table FM.A-4, Objective 5: Low-level requirements conform to standards

- FM.6.3.2.e. Conformance to standards

Specific standards have been defined for formal LLR, conformance to these standards are verified using reviews and automated checks by the theorem proving tool syntax analyzer.

Table FM.A-5, Objective 1: Source Code complies with low-level requirements

- FM.6.3.4.a. Compliance with low-level requirements

The compliance of the source code with the LLR is achieved through the use of the theorem proving tool which verifies that the source code satisfies the properties.

- FM.6.3.4.c. Verifiability

The Source Code has mathematically defined syntax and semantics and is verified using formal methods. Moreover, specific coding standards are defined for the code that is verified by formal proof in order to ensure that the code will be compatible with the verification tool.

- FM.6.3.4.e. Traceability

The verification by the theorem proving tool that the Source Code satisfies all the properties which are the LLRs ensures that the software LLRs were developed into Source Code.

Table FM.A-5, Objective FM 10: Formal analysis cases and procedures are correct

- FM.6.3.6.a. Formal analysis cases

In this case, the formal analysis cases are the properties, that is to say, they are the LLRs. So it is straightforward to see that we have analysis cases for each LLR.

- FM.6.3.6.b. Formal analysis procedures

This objective is achieved by reviews. These reviews are performed to ensure that the defined properties are precise, the environment is representative, and the proof methodology is well defined.

Table FM.A-5, Objective FM 11: Formal analysis results are correct and discrepancies explained

- FM.6.3.6.c. Formal analysis results

A review is performed to ensure that the results conform to what was expected.

Table FM.A-6, Objective 3: Executable Object Code complies with low-level requirements

- FM.6.7.d. Compliance with low-level requirements

Formal analysis of Source Code is performed to reach this objective, with complementary analyses to show property preservation between Source Code and object code.

Table FM.A-6, Objective 4: Executable Object Code is robust with low-level requirements

- FM.6.7.b. Robustness with low-level requirements

Formal analysis of Source Code is performed to reach this objective, with complementary analyses to show property preservation between source code and object code.

Table FM.A-7, Objective FM 4: Coverage of low-level requirements is achieved

- FM.6.7.1.1.a. Formal analysis cases exist for each software requirement

The properties are both the requirements and the verification cases, and therefore there is a verification case for every requirement. This is an advantage of using formal methods.

- FM.6.7.1.1.b. Formal analysis cases satisfy the criteria of normal range and robustness criteria

This is satisfied by the definition of the properties. In the properties, normal and robustness execution conditions are defined and the behavior of the software is given for each execution condition.

- FM.6.7.1.1.c. Any deficiencies identified in the analysis are resolved.

There are no deficiencies, because satisfaction of FM.6.7.1.1.a means that the set of cases is complete, and satisfaction of FM.6.7.1.1.b means that the cases satisfy the normal range and robustness criteria.

Table FM.A-7, Objective FM 5-8: Verification of software structure is achieved

- FM.6.7.1.2 Complete coverage of each requirement

Each requirement is expressed as a property, and each property is formally proved exhaustively, so this objective is achieved.

- FM.6.7.1.3 Completeness of the set of requirements

Compliance with the objective of completeness of properties is verified through the following elements:

1. The data flow verification gives evidence that the data used by the Source Code are those defined at the design level.

This is done by the theorem proving tool analyzing the Source Code to extract its data flow and comparing it to the data flow defined at the design level.

And:

2. Verification of the completeness of the defined conditions by the use of “Always” properties:

-- Completeness of the conditions

Always COND0: $\text{COND_ERR} \vee \text{COND_FCT}$;

There is no other case apart from these two

This is done by the theorem proving tool.

3. Verification, by review, that an accurate property exists and specifies the value of each Output for each execution condition.

- FM.6.7.1.4 Detection of unintended dataflow relationships

The data flow verification gives evidence that the operands used by the Source Code are those defined at the design level.

This is done by the theorem proving tool analyzing Source Code to extract its data flow and comparing it to the data flow defined at the design level.

A manual review is performed to verify that an accurate property exists and specifies the value of each Output for each execution condition.

- FM.6.7.1.5 Detection of extraneous code, including dead code, and deactivated code

There is no deactivated code. Otherwise it would have been specified (LLR) by properties and proved as well.

The theorem proving tool will not be able to detect unreachable code: all the properties defined for the function will be proved. However, note that the completeness of the properties and the exhaustiveness of the formal proof guarantee that if any code section is a dead code section, it will have no impact on the results of the function whatever the execution of the function is.

Identification of extraneous code, including dead code, was undertaken by an independent manual review of the Source Code focused on detection of unreachable code.

Table FM.A-7, Objective FM 9: Verification of property preservation between source and object code

- FM.6.7.f. Verification of property preservation between source and object code

An analysis of the object code generated by the compiler including the effect of the compiler options on the object code is performed to ensure that this objective is achieved.

FM.B.1.5.2 Example 2: Verification of Model-Based Software Design

In this example, the application of formal methods to a model-based software development process is shown.

- HLRs are initially expressed as English “shall” statements that are subsequently formalized for analysis.
- LLRs are software models developed using model-based design tools for capturing data flow diagrams and state machines.
- The LLR models are analyzed using a model checker to verify whether or not they satisfy the HLRs.
- Source code is automatically generated from the LLRs and tested in conformance with a conventional test-based process.

The primary objective in this application of formal methods was to identify and remove design errors early in the development process. The comprehensive analysis provided by model checking provides a higher assurance of correct behavior than test-based verification. Detection and correction of errors earlier in the development process (during design rather than test and integration) reduces development cost.

A unique feature of the methodology employed is the use of an automated translation framework to convert software models to formal notations that can be analyzed by model checkers. Furthermore, counterexamples identified by the model checker can be simulated step by step in the original development environment. This permitted engineers to use development tools and models with which they were already familiar.

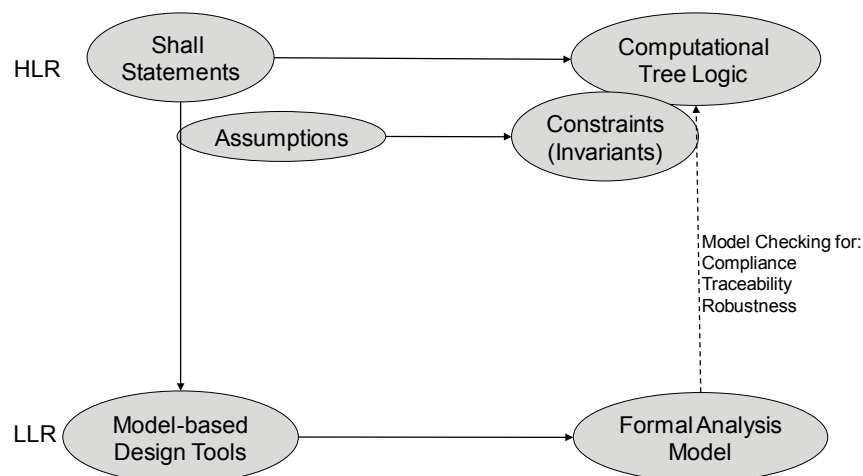


Figure FM.B.1-3 Overview of Requirement Artifacts

This example will emphasize the formalization of the software requirements and describe the process used to verify their implementation using model checking.

Overview of Window Manager Application

In a modern aircraft, the primary way that aircraft status information is displayed to pilots is through computerized display panels. These display panels are designed to replace the dozens of mechanical switches and dials found in earlier aircraft and to present a unified and straightforward interface to critical flight information. The display panels are configurable to allow pilots to select different information for display, including navigational maps, aircraft system status, and flight checklists. However, some information is considered critically important and must always be displayed.

The Window Manager (WM) application is responsible for determining what information is displayed to the pilots at any time. A simplified architecture for an aircraft display system is presented in [Figure FM.B.1-4](#). In this figure, two Display Units (DU) provide information to the pilot and co-pilot. These displays are connected to two redundant computing resources via a high-availability bus. Each computing resource runs several display applications (PFD, EICAS, and MAP, in this example) and a window manager, and is capable of managing both DUs. Each display has a “preferred” computing resource that it uses if both resources are available. The pilots can interact with the displays through the use of a keypad and a trackball (not shown in the figure). The trackball controls a cursor that is visible on the displays. The cursor can move between displays in response to pilot actions.

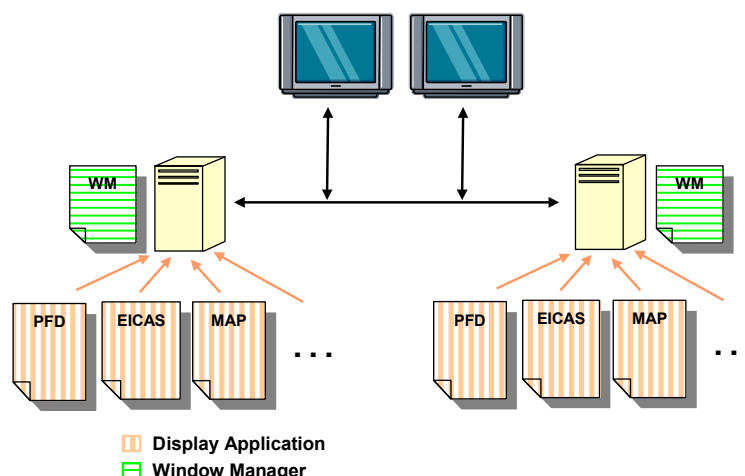


Figure FM.B.1-4 Simplified Display System Architecture

The WM determines which applications should be displayed on which display area as well as the location of the cursor on the displays. It has several responsibilities related to routing information to the displays. First, the WM must update which applications are being displayed in response to user selections of display applications. Second, the WM must handle reversion in case of hardware or application failures. If a display fails, the WM decides which information is most critical and moves this information to the remaining display. Another responsibility has to do with cursor management: some display applications support the cursor while others do not. It is the responsibility of the WM to ensure that the cursor does not appear on a display that contains an application that does not support the cursor. In the event of reversion, the WM must ensure that the cursor is not tasked to a dead display.

The WM is essential to the safe flight of an airplane. If a WM contains logic errors, it is possible that critical flight information will be unavailable to the flight crew. Hence it is required to meet the Level A objectives of DO-178.

Window Manager High-level Requirements

A simplified WM application consists of two major subsystems. The `DU_APPLICATION_SELECTION` subsystem handles mapping applications to display units, and `CURSOR_PLACEMENT` handles the location of the cursor.

The inputs to the WM application are:

- **Left/Right DU Available:** These inputs describe whether or not the left (right) display unit is available.
- **Left/Right Switchbanks:** The pilot and copilot have three-way switches to select which application to display on a display unit. The three switch positions are EICAS, PFD, and MAP. Depending on the switch position, one of the left (right) switch bank inputs is true and the other left (right) switch bank inputs are false. When the switch is being turned, it is also possible that all of the switch bank inputs are false.

- Left/Right Cursor Manual Requests: The pilot or copilot can manually request that the display cursor be moved to the left display or the right display using two buttons. In these instances, the left or right manual request inputs are true.

The outputs of the WM application are:

- Left/Right DU Application: These outputs are enumerations: {0 = BLANK, 1 = EICAS, 2 = PFD, 3 = MAP} that describe what is to be displayed on a DU.
- Cursor Location: This output is an enumeration {0 = No Cursor, 1 = Left DU, 2 = Right DU} that describes the location of the display cursor.

A typical software requirements document will specify requirements informally, using English “shall statements. For our WM example, we will consider the following subset of the requirements:

R1: If a DU is available, then it shall display some application.

R2: If a DU is unavailable, then it shall not attempt to display any application.

R3: The cursor shall not be displayed on a DU that is unavailable.

R4: The cursor shall not be displayed on a DU whose application is not MAP.

R5: When the MAP application is selected for a DU then the cursor shall move to that DU.

R6: When the EICAS button is pressed, the display application shall be EICAS.

R7: When the PFD button is pressed, the display application shall be PFD.

R8: When the MAP button is pressed, the display application shall be MAP.

Turning the High-Level Requirements into a Formal Notation

These requirements must then be formalized as properties; that is, they must be expressed in a precise syntax that can be analyzed by a model checker. The model checking tool used in this example analyzed properties specified in Computation Tree Logic (CTL). A complete introduction to CTL (and some other formal specification languages) can be found in [Cla01]. For a synchronous system where the requirements are specified as “shall” statements over system inputs and outputs, this process is relatively straightforward as most requirements fall into one of a handful of patterns. In [Mil03], the process of translating these informal statements into safety properties is described in more detail.

The software requirements document is not the only source of properties to be analyzed. Properties also emerge from discussions with developers about the functionality of different subsystems, or even from a careful review of a particular implementation detail of the model. In some cases, these properties can be thought of as validity checks for particular implementation choices, but on occasion they lead to additions to the requirements document.

For example, in the WM model, the LEFT_DU_APPLICATION and RIGHT_DU_APPLICATION outputs are implemented as integers, but they are supposed to behave as enumerations. Therefore, they should always be in the range 0 (BLANK) to 3 (MAP). Otherwise, the output is out-of-range. This aspect of the implementation can be easily added as a new requirement and formalized.

R9: Each DU_APPLICATION variable shall always be assigned a value between 0 and 3

Most of the requirements in the WM fall into two CTL patterns. The first is simply a constraint that must be maintained by all reachable states. For example, the requirement R1 can be translated into two CTL properties (one for each side) as follows:

```
AG(LEFT_DU_AVAILABLE -> LEFT_DU_APPLICATION != BLANK )
AG(RIGHT_DU_AVAILABLE -> RIGHT_DU_APPLICATION != BLANK )
```

The AG operator states that the property must hold for all reachable states, and -> is the implication operator. The formal property for the left side can be paraphrased:

In all reachable states, if the left DU is available, then its application shall not be blank.

The second pattern is a constraint over a reachable state and all possible next states. This template is required when we are interested in observing the behavior of the system in response to some event. For example, in requirement R5 we are interested in the moment when the MAP application is selected. To specify what we mean by “selected,” we have to describe the change in state of LEFT_DU_APPLICATION from “not MAP” to “MAP.” The AX, or “all next states” operator provides this ability.

Requirement R5 can be translated into CTL (for the left side) as:

```
AG( LEFT_DU_APPLICATION != MAP ->
    AX( LEFT_DU_APPLICATION = MAP ->
        CURSOR_LOCATION = LEFT_DU ) )
```

where the AX operator encloses a property that must hold in all next states. Given these operator definitions, the property can be paraphrased:

In any state in which LEFT_DU_APPLICATION is not equal to MAP, then in any next state in which LEFT_DU_APPLICATION is equal to MAP, the CURSOR_LOCATION must be LEFT_DU.

Given the system inputs described in [Figure FM.B.1-4](#) we can formalize the WM requirements in CTL as shown in [Table FM.B.1-2](#). Independent manual reviews of the requirements and properties were conducted to ensure that the requirements were correctly formalized.

Table FM.B.1-2 Window Manager Requirements Formalized as CTL

Requirement	CTL Properties
R1: If a DU is available, then it shall display some application	AG(LEFT_DU_AVAILABLE -> LEFT_DU_APPLICATION != BLANK) AG(RIGHT_DU_AVAILABLE -> RIGHT_DU_APPLICATION != BLANK)
R2: If a DU is unavailable, then it shall not attempt to display any application	AG(!LEFT_DU_AVAILABLE -> LEFT_DU_APPLICATION = BLANK) AG(!RIGHT_DU_AVAILABLE -> RIGHT_DU_APPLICATION = BLANK)
R3: The cursor will not be displayed on a DU that is unavailable	AG(!LEFT_DU_AVAILABLE -> CURSOR_LOCATION != LEFT_DU) AG(!RIGHT_DU_AVAILABLE -> CURSOR_LOCATION != RIGHT_DU)
R4: The cursor shall not be displayed on a DU whose application is not MAP	AG(LEFT_DU_APPLICATION != MAP -> CURSOR_LOCATION != LEFT_DU) AG(RIGHT_DU_APPLICATION != MAP -> CURSOR_LOCATION != RIGHT_DU)
R5: When the MAP application is selected for a side, then the cursor shall move to that side	AG(LEFT_DU_APPLICATION != MAP -> AX(LEFT_DU_APPLICATION = MAP -> CURSOR_LOCATION = LEFT_DU)) AG(RIGHT_DU_APPLICATION != MAP -> AX(RIGHT_DU_APPLICATION = MAP -> CURSOR_LOCATION = RIGHT_DU))
R6: When the EICAS button is pressed (SELECT_EICAS), EICAS shall be the display application	AG(LEFT_SELECT_EICAS -> LEFT_DU_APPLICATION = EICAS) AG(RIGHT_SELECT_EICAS -> RIGHT_DU_APPLICATION = EICAS)
R7: When the PFD button is pressed (SELECT_PFD), PFD shall be the display application	AG(LEFT_SELECT_PFD -> LEFT_DU_APPLICATION = PFD) AG(RIGHT_SELECT_PFD -> RIGHT_DU_APPLICATION = PFD)
R8: When the MAP button is pressed (SELECT_MAP), MAP shall be the display application	AG(LEFT_SELECT_MAP -> LEFT_DU_APPLICATION = MAP) AG(RIGHT_SELECT_MAP -> RIGHT_DU_APPLICATION = MAP)
R9: Each DU_APPLICATION variable shall always be assigned a value between 0 and 3.	AG(LEFT_DU_APPLICATION <= 3 & LEFT_DU_APPLICATION >= 0) AG(RIGHT_DU_APPLICATION <= 3 & RIGHT_DU_APPLICATION >= 0)

Window Manager Low-Level Requirements

The low-level requirements for the Window Manager application are captured in a modeling tool as a combination of dataflow diagrams and state machines. The top level software model is as follows:

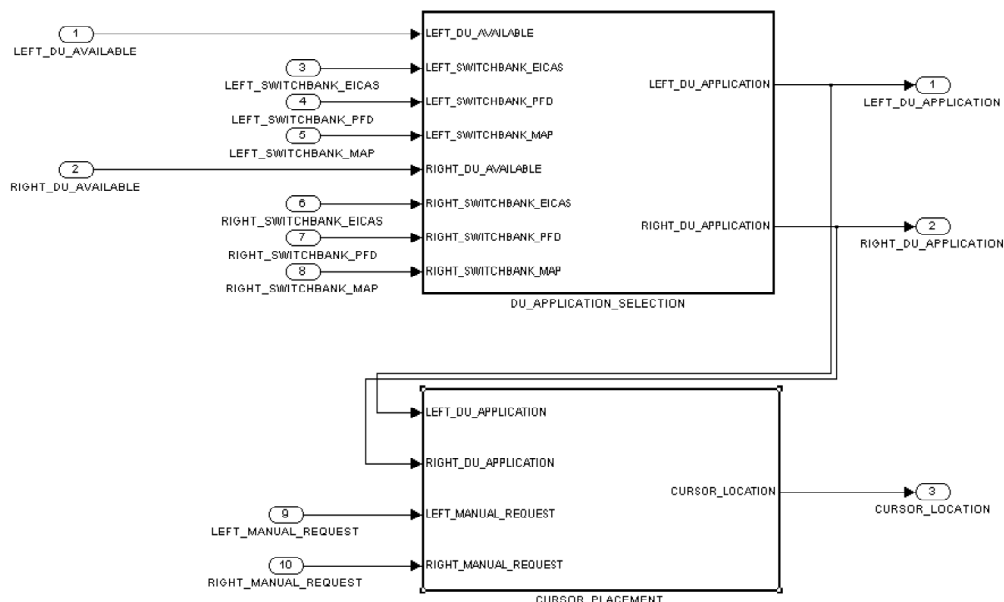


Figure FM.B.1-5 Window Manager Top Level Software Model

Turning the Low-Level Requirements into a Formal Notation

Analysis of the software design models by a symbolic model checking tool has been automated by means of a translation framework. Models are automatically transformed in a series of small semantics-preserving translation steps. The translation framework supports a variety of model description formats and backend analysis tools, including both model checkers and theorem provers.

Formal Analysis of LLRs against HLRs

In this example, a symbolic model checker was used to verify the WM properties. This model checker utilizes both Binary Decision Diagram (BDD) and SAT-based (satisfiability) model checking algorithms, which are known to be sound. Properties to be verified in the symbolic model checker are specified using either Computation Tree Logic (CTL) or Linear Temporal Logic (LTL).

The advantage of using a model checker is that it will check all possible combinations of inputs and state to determine if a property is true. The model checker has been used to verify properties of models with over 10^{120} reachable states.

It is often not possible to verify properties on a large model in a completely unconstrained environment. Instead, the correctness of the formal analysis depends on certain assumptions about the systems environment. As part of the analysis process, we examine the assumptions in the requirements document to create constraints on the

possible values of inputs into the system. The constraints on inputs are specified as invariants, which restrict the states that will be visited during the model checking process. In this step, we determine and formalize the applicable assumptions for each subsystem as invariants.

As the requirements in this case are informal and so are the assumptions there is potential for them to be invalid or for them to conflict. These errors can cause the analysis to be unsound. Therefore, although we formalize and capture the assumptions as part of the verification process, we do not use them in our initial model checking analysis. If the initial analyses return counterexamples, we evaluate the counterexamples to see whether they are due to violations of our assumptions or due to incorrect behavior of the LLR model. Even if counterexamples are due to violations, it may be preferable to strengthen the model behavior, when possible, to deal with abnormal conditions rather than use system assumptions. If it is determined that there is no good way to handle the abnormal condition within the model, then we use the assumptions to attempt to constrain the environment such that no counterexamples can be found. In this case, the assumptions used should be verified against the system requirements. If counterexamples are found that cannot be removed by modifications to the LLR model or constraining the environment then these constitute errors in the HLR.

The overall analysis process is illustrated in [Figure FM.B.1-6](#). Software engineers use the software requirements (or “shall” statements) to develop a model of the system. Independently, the requirements are expressed as CTL properties. The translation framework automatically generates an analysis model, merges the CTL properties, and analyzes them using the model checker. If a property does not hold, the model checker generates a counterexample that demonstrates the error condition.

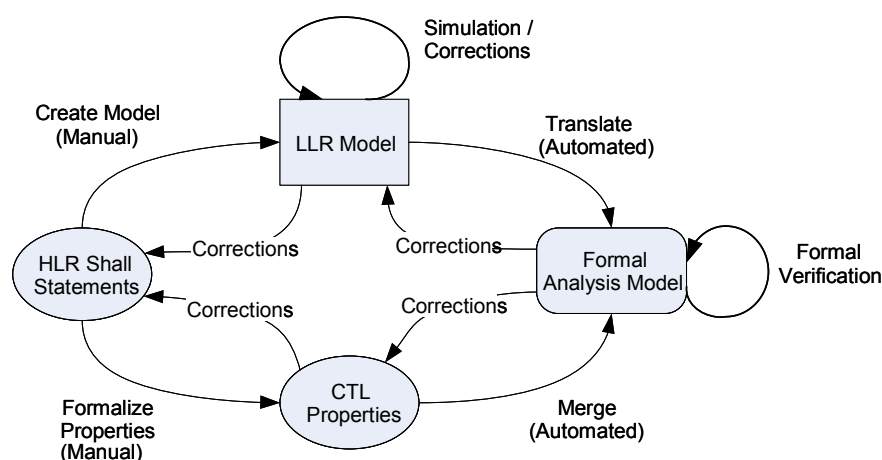


Figure FM.B.1-6 Analysis Process

When a counterexample is discovered, the error is classified by its underlying cause and corrected. The cause may be one or more of the following:

- Modeling error.
- Property formalization error.

- Incorrect/missing environmental constraints (invariants).
- Requirements error.

Analysis Results

The actual WM is much larger and more complex than the simplified window manager described here. Approximately 90% of the functional behavior of the WM application (in terms of the number of design blocks in the model) was verified using model checking. The remaining 10% of the model is in one subsystem that contains a significant number of real- and integer-valued variables. This subsystem does not contain much mode-specific behavior and was verified using conventional methods.

The WM was divided into five subsystem models that are used for analyzing its behavior. Table FM.B.1-3 provides an overview of these subsystems and the analysis results.

Table FM.B.1-3 Window Manager Analysis Data

Subsystem	Model Diagrams	Basic Blocks	State Space	Properties	Errors found
GG	2,831	10,669	9.8×10^9	43	56
PS	144	398	4.6×10^{23}	152	10
CM	139	1,009	1.2×10^{17}	169	10
DUF	879	2941	1.5×10^{37}	115	8
MFD	302	1,100	6.8×10^{31}	84	14
Totals	4295	16,117	n/a	563	98

The above results show that formal analysis can be applied to large systems. In each case 100% of the subsystem behavior is analyzed by the model checker. The size of each of these subsystems state space is shown in the table. This example does not address the completeness of HLR coverage although further verification can achieve this. The 98 errors found resulted in changes to the LLR model or changes to the HLRs. The corrected HLRs and LLRs were re-analyzed and found to be compliant.

Certification Objectives

The WM effort was conducted several years ago, before any consideration of incorporation of formal methods guidance in DO-178. The two objectives of the application of formal methods were to

1. increase our confidence in the software.
2. reduce development and verification effort by detecting and correcting errors in the design phase.

The formal methods technology developed in the project was successfully transitioned to the product development organization. By the end of the project, all analysis work was being performed by product development software engineers, with minimal assistance from researchers.

With the new guidance provided in the Formal Methods Supplement to DO-178C, many certification objectives could have been satisfied although functional tests executed in target hardware would always be required to establish correctness of the overall system. Some examples follow of objectives that could have been satisfied by the formal method.

- FM.6.2.1 Considerations for Formal Methods
 - a. Formal notations: Properties to be verified were specified in CTL. Formal definition of CTL may be found in [Em90] and [Hu04]. The models were given formal definition through the translation process, which includes a formal syntax and translation rules for each model element.
 - b. Soundness: The BDD and SAT algorithms are known to be sound. Details of the BDD algorithm used for model checking and its soundness can be found in [Mc93] and [Hu04]. Application of satisfiability solving to the model checking problem and its soundness is described in [CBRZ01].
 - c. Assumptions: All assumptions necessary for the analysis were justified through the use of invariants derived from system requirements.
- FM.6.3 Software Reviews and Analysis
 - i. Requirement formalization correctness: In this project, all requirements were captured and managed using a requirements-management tool. For each requirement, the corresponding formalization was captured in the requirements-management tool with one or more CTL statements. Independent manual reviews were conducted to ensure that the CTL statements accurately described the original English-language requirement.
- FM.6.3.1 Reviews and Analyses of High-Level Requirements
 - d. Verifiability of HLR: The ability to express the high-level requirements for the system in CTL is a sufficient demonstration of verifiability in this example.
 - e. Conformance to standards: Requirements that do not conform to the standard for CTL syntax will be identified and rejected by the analysis tools. This feature of the tool would need to be qualified. Alternatively, conformance to CTL syntax can be easily checked by a manual review.
- FM.6.3.2 Reviews and Analyses of Low-Level Requirements
 - a. Compliance with HLR: Analysis by model checking demonstrated that low-level requirements (the system model) complied with high-level requirements. This feature of the tool would need to be qualified.

FM.B.1.6 Summary

Formal methods were developed as a branch of computer science in order to reason more scientifically about software. Initially the advantages of formal methods were in analyzing the behavior of source code to understand where this was incorrect. Since those early applications of this approach in the 1970's the problem of error prone source code

has reduced and instead most of the errors in software development are now generally accepted as being attributable to errors in requirements (LLR or HLR).

It has become apparent that creating formal models of requirements can help to address this problem in the same way as it did with source code. A further advantage of using formal methods at the requirements level is the ability to derive or refine from these requirements the code itself, thus ensuring that no errors are introduced at this stage. Alternatively their use at the requirements level allows formal analysis between requirements and code to establish correctness.

The current perception of the limitations of formal methods at this time fall into three main categories:

- “State-space” or complexity explosion of most software systems. Formal methods consider all possibilities when proving properties and therefore the size of the code or requirement set for which properties can be proved is limited. The result of this limitation is that proofs are often carried out on relatively small software components and the verification evidence is built up or assumptions are made about the system environment to make the analysis feasible.
- Education. The industry is predominately made up of engineers rather than computer scientists and the formal notations and formal proofs are often an alien concept. In comparison, testing is a very tangible engineering discipline understood in most if not all engineering fields.
- Theory versus reality. Engineers are generally less comfortable with theoretical deduction compared to physical demonstration. With formal methods there is a balance point between the completeness of a theoretical assertion and the incomplete, but practical, demonstration achieved with testing.

The complexity explosion has been greatly improved and will further be improved in the future by growth in computing power and by research into the simplification of the proofs themselves.

The education issue should not be a difficult problem to solve but there seems to have been a large gap between the academic community researching formal methods and the industrial applicants who might benefit. Industry needs to be educated, but also the academic world may need to better understand the practical issues with applying the theories. However the approach is not at odds with other engineering disciplines, it is just that the mathematical techniques are different. Consider a civil engineer building bridges. He does not rely on testing as verification of his design. He uses well established science and mathematics to reason about it before it is built.

Probably the hardest issue to overcome is the gap between the thoroughness of theoretical deduction and the practical demonstration of test run in the target hardware. It is true that in formal methods assumptions must be made in order to prove things, such as how the program will actually execute on the hardware. Therefore it may be the case that because of a false assumption erroneous behavior is only discovered when the code is run in target hardware. However, there are features of the software behavior which can be found through formal methods which are unlikely to ever be detected by testing. There are a number of in-service products that have subsequently undergone formal analysis and undesirable behavior has been identified.

One way in which this gap may be closed in the future is by the use of formal models of the hardware itself, preferably the same models that were used to create the complex hardware devices and used, for example, in their certification.

So when formal methods is used to its maximum effect at some point in the future the software life cycle might look like this:

- The applicant captures their customer's requirements using some kind of formal notation that immediately forces removal of ambiguity, standardization in format and clarity of understanding for the systems engineers.
- The system engineers take these requirements and match up formal specifications of existing sub-components or they create high-level requirements in a formal notation. In both cases a formal analysis is carried out to ensure that the sub-components or sub-systems meet the customer requirements.
- The software engineers then implement an architecture and designs to satisfy the high-level requirements using formal notations. These designs are analyzed formerly against the high-level requirements.
- Code is auto-generated or refined from the architecture/design or manually coded. If manually coded, a formal analysis is performed to show that it meets the design.
- A formal model of the target hardware is used to ensure that the analysis of the code takes into account the complete behavior of the target.
- The code is integrated and a set of tests run to establish confidence in the final system.
- The system is delivered to the customer.

It is apparent from this discussion paper that formal methods are in use today to develop safety-critical systems. Research is on-going to develop industrial strength tools and techniques to enable wider application. The Formal Methods Supplement to DO-178C provides a guide for the claims of credit for certification through the use of these techniques.

FM.B.1.7 References

- [Cla01] E. Clarke, O. Grumberg, and P. Peled, Model Checking, The MIT Press, Cambridge, Massachusetts, 2001.
- [Fam99] Famantanantsoa Randimbivololona, Jean Souyris, Patrick Baudin, Anne Pacalet, Jacques Raguideau, Dominique Schoen: Applying Formal Proof Techniques to Avionics Software: A Pragmatic Approach. World Congress on Formal Methods 1999: 1798-1815.
- [Hil50] David Hilbert and Wilhelm Ackermann 1950. Principles of Mathematical Logic (English translation). Chelsea. The 1928 first German edition was titled Grundzüge der theoretischen Logik.
- [Hoa69] C. A. R. Hoare. An axiomatic basis for computer programming. Communications of the ACM, 12(10):576–580,583 October 1969.
- [Mil09] S. Miller, “Bridging the Gap Between Model-Based Development and Model Checking,” Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2009), LNCS 5505.
- [Mil03] S. Miller, M. Heimdahl, and A. Tribble, “Proving the Shalls,” Proceedings of FM 2003: the 12th International FME Symposium, Pisa, Italy, Sept. 8-14, 2003.
- [Sou09] Jean Souyris, Virginie Wiels, David Delmas, Hervé Delseny: Formal Verification of Avionics Software Products. FM 2009: 532-546.
- [Wha07] M. Whalen, D. Cofer, S. Miller, B. Krogh, W. Storm, “Integration of Formal Analysis into a Model-Based Software Development Process,” Formal Methods for Industrial Critical Systems (FMICS 2007), LNCS 5596.